



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

严谨 · 勤奋 · 求实 · 创新

数字系统设计

09-工具链优化与存储器结构

贾弘洋 & 刘勇攀

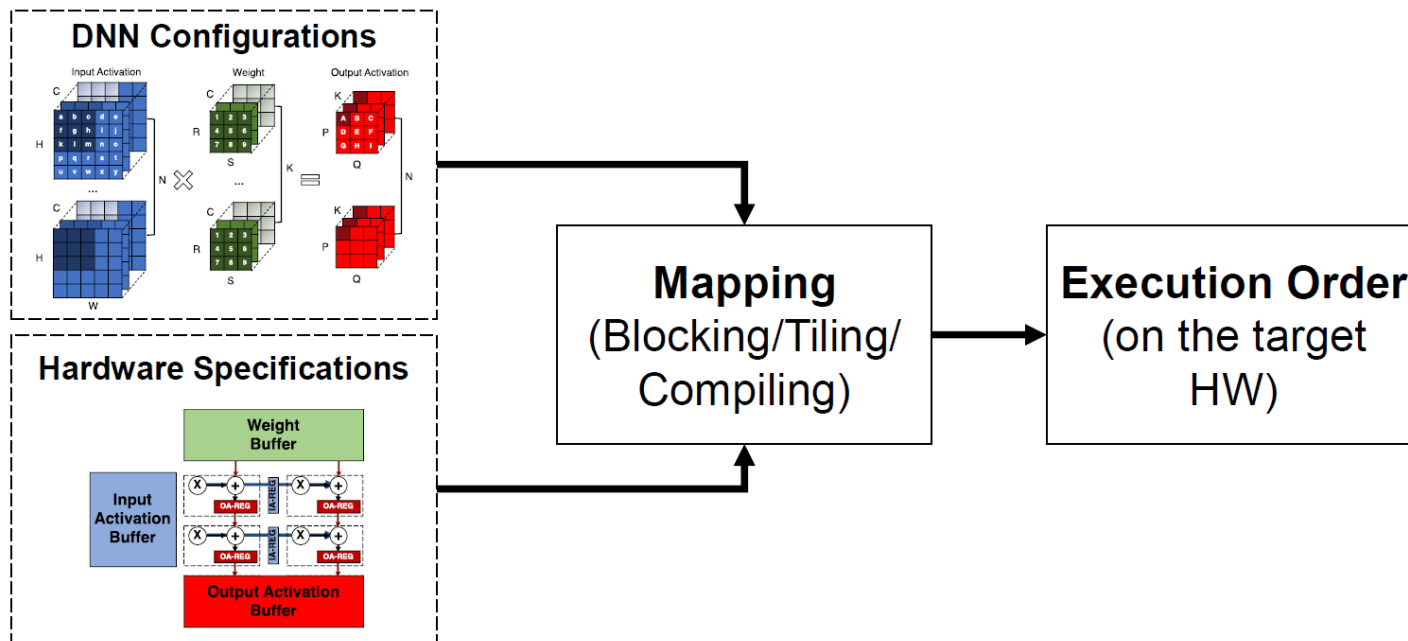
2025/4/15



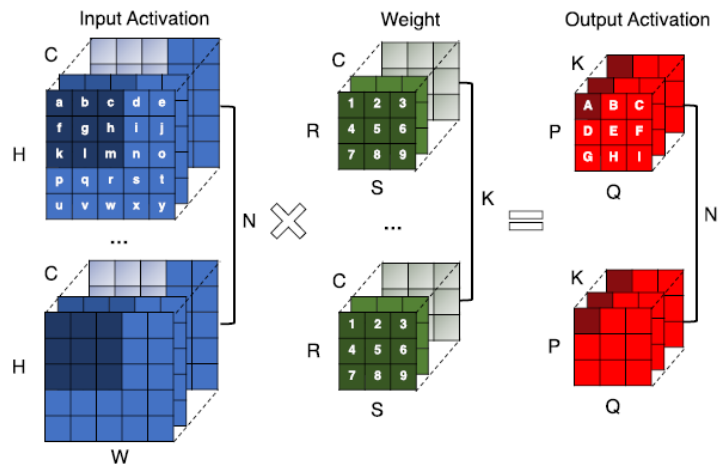
/ Review: 神经网络的映射问题



Mapping: Large network model $\rightarrow$ limited hardware resources.
Reducing additional data movement/recompute overheads.



/ Review: CNN的循环嵌套 (Loop Nest) 表述

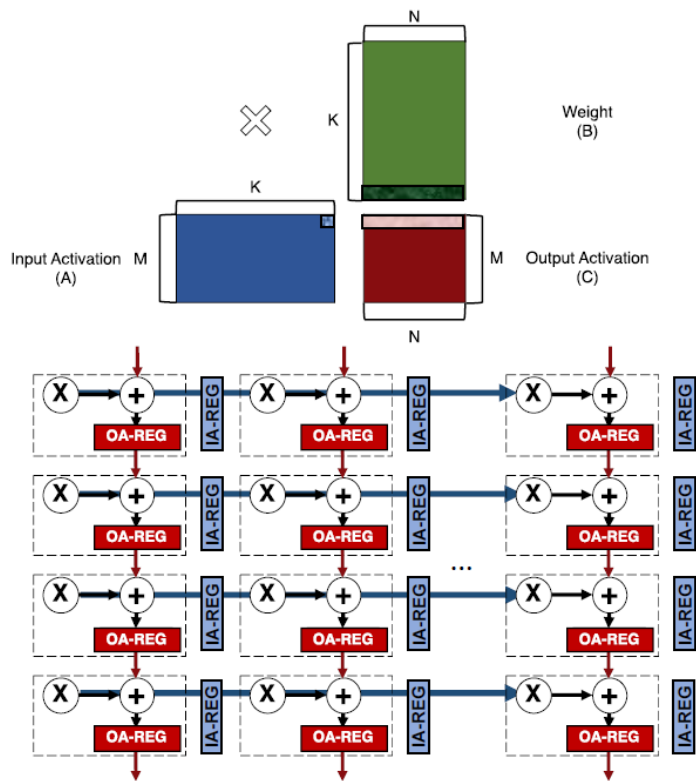


```
for (n=0; n<N; n++) {  
  for (k=0; k<K; k++) {  
    for (p=0; p<P; p++) {  
      for (q=0; q<Q; q++) {  
        OA[n][k][p][q] = 0;  
        for (r=0; r<R; r++) {  
          for (s=0; s<S; s++) {  
            for (c=0; c<C; c++) {  
              h = p * stride - pad + r;  
              w = q * stride - pad + s;  
              OA[n][k][p][q] +=  
                IA[n][c][h][w]  
                * W[k][c][r][s];  
            }  
          }  
        }  
        OA[n][k][p][q] = Activation(OA[n][k][p][q]);  
      }  
    }  
  }  
}
```

for each output activation

convolution window

Review: Loop Nest的空间展开 (以TPU为例)



```
for (m=0; m<M; m++) {  
  spatial_for (n=0; n<N; n++) {  
    OA[n,m] = 0;  
    spatial_for (k=0; k<K; k++) {  
      OA[n,m] += IA[m, k]  
                * W[k, n];  
    }  
    OA[n,m] = Activation(OA[n,m]);  
  }  
}
```

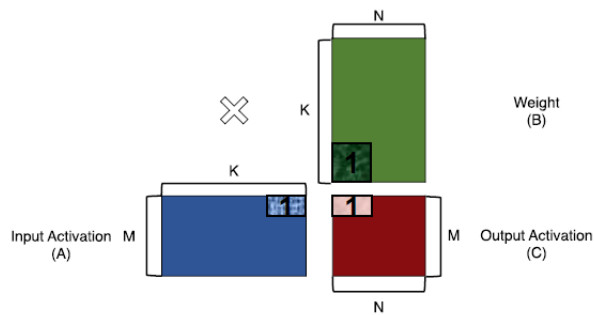
- Systolic accumulation
- Systolic multicast

/ Review: Q1: What if the systolic array size $< K$ or N ?



Can not mapping the all weights to the systolic array in the same time.

Adding two bounds, i.e., $K1$ and $N1$, to mapping part of the weights.

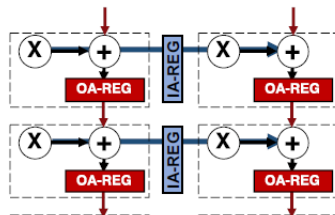


- Hardware Specifications:

- Systolic array size: 2×2

- Notation:

- $K?/N?$: loop bounds

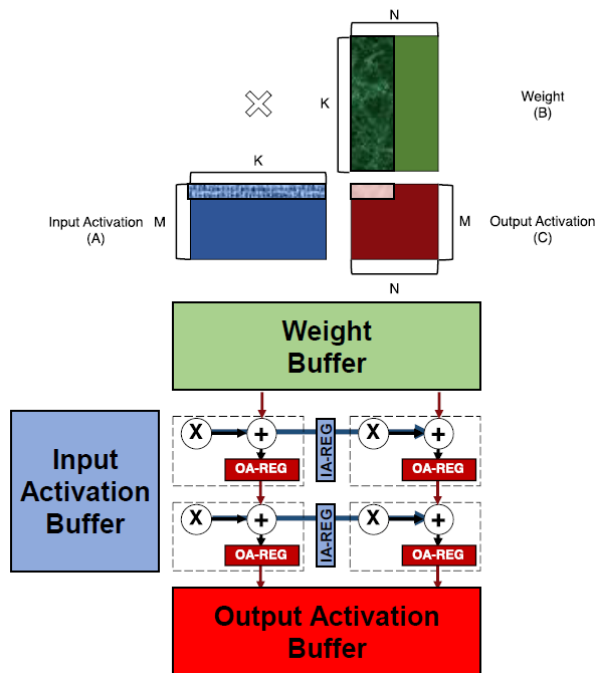


```
for (m=0; m<M; m++) {  
  for (n1=0; n1<N1; n1++) {  
    OA[n1*N0:(n1+1)*N0,m] = 0;  
    for (k1=0; k1<K1; k1++) {  
      spatial_for (n0=0; n0<N0; n0++) {  
        spatial_for (k0=0; k0<K0; k0++) {  
          OA[n1*N0+n0,m] += IA[m, k1*K0+k0]  
            * W[k1*K0+k0, n1*N0+n0];  
        }  
      }  
    }  
  }  
}
```

/ Review: Q2: What if weight buffer size $< N \times K$?



Weights need to be move in/out to the weight buffer. Optimize to minimize data movements.

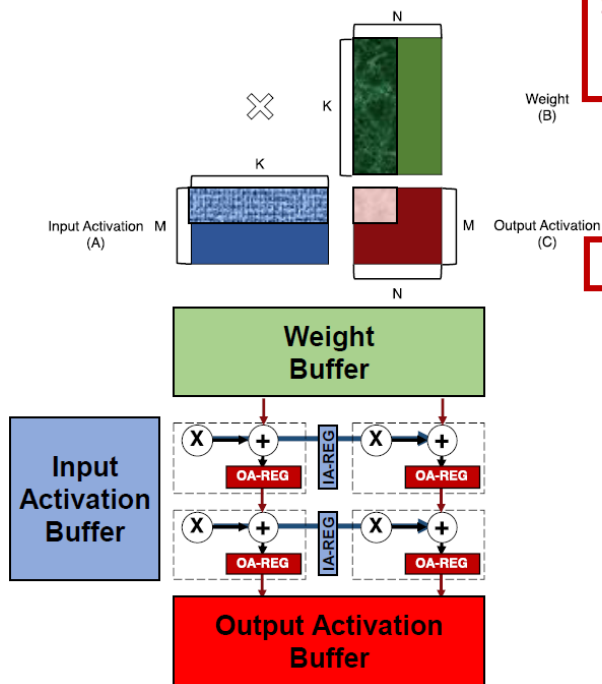


```
for (m=0; m<M; m++) {  
    mvin(IA[m:m+1,0:K]);  
    for (n2=0; n2<N2; n2++) {  
        // W buffer stores:  $N1 \times N0 \times K < NK$   
        mvin(W[0:K,n2*N1*N0:(n2+1)*N1*N0]);  
        OA[n2*N1*N0:(n2+1)*N1*N0,m:m+1]=0;  
        for (n1=0; n1<N1; n1++) {  
            for (k1=0; k1<K1; k1++) {  
                spatial_for (n0=0; n0<N0; n0++) {  
                    spatial_for (k0=0; k0<K0; k0++) {  
                        OA[n2*N1*N0+n1*N0+n0,m]  
                            +=IA[m, k1*K1+k0]  
                            * W[k1*K0+k0,n2*N1*N0+n1*N0+n0];  
                    }  
                }  
            }  
        }  
        mvout(OA[n2*N1*N0:(n2+1)*N1*N0,m:m+1]);  
    }  
}
```

/ Review: Q3: What if input buffer size $> 1 \times K$?



Should store >1 rows of input activations.



```
for (m2=0; m2<M2; m2++) {  
    // IA buffer stores:  $M1 \times K$   
    mvin(IA[m2*M1:(m2+1)*M1, 0:K]);  
    for (n2=0; n2<N2; n2++) {  
        // W buffer stores:  $N1 \times N0 \times K$   
        mvin(W[0:K, n2*N1*N0:(n2+1)*N1*N0]);  
        OA[n2*N1*N0:(n2+1)*N1*N0, m2*M1:(m2+1)*M1] = 0;  
        for (m1=0; m1<M1; m1++) {  
            for (n1=0; n1<N1; n1++) {  
                for (k1=0; k1<K1; k1++) {  
                    spatial_for (n0=0; n0<N0; n0++) {  
                        spatial_for (k0=0; k0<K0; k0++) {  
                            OA[n2*N1*N0+n1*N0+n0, m2*M1+m1]  
                                += IA[m2*M1+m1, k1*K1+k0]  
                                    * W[k1*K0+k0, n2*N1*N0+n1*N0+n0];  
                        }  
                    }  
                }  
            }  
        }  
        mvout(OA[n2*N1*N0:(n2+1)*N1*N0  
            , m2*M1:(m2+1)*M1]);  
    }  
}
```

/ Review: Mapping Dimensions



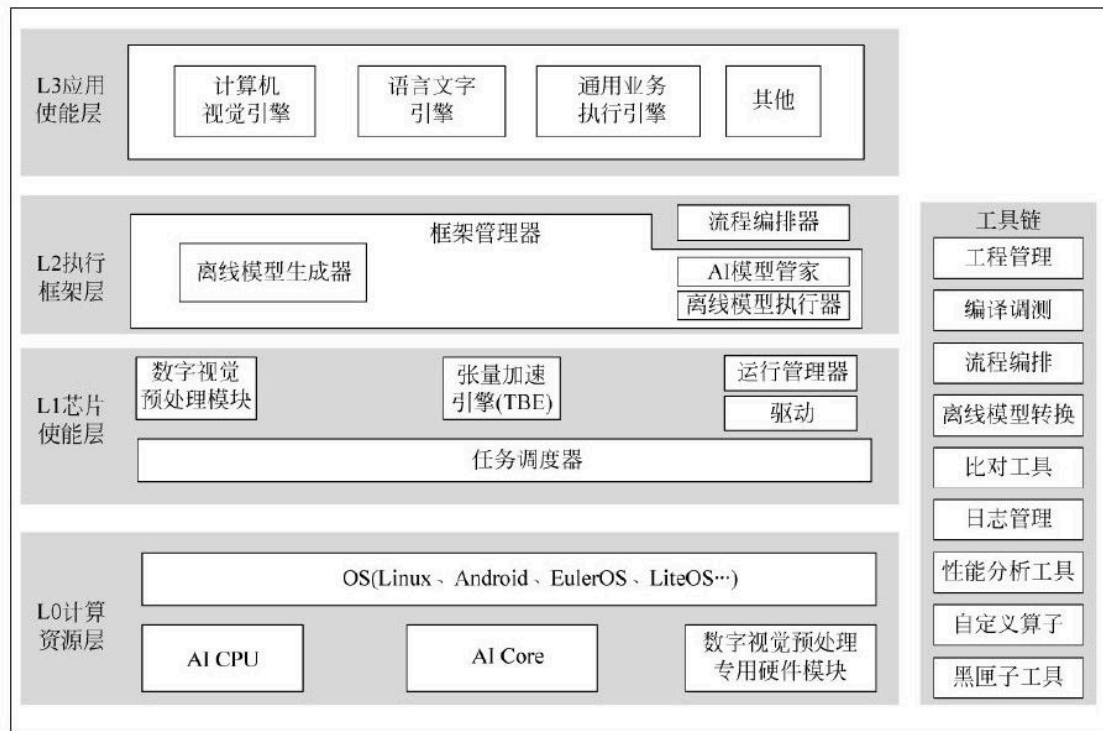
- Loop ordering:
 - Which index goes to the inner/outer loop?
- Loop bounds:
 - What are the loop bounds (i.e., $N?$ / $K?$ / $M?$) for each loop?
- Spatial choice:
 - Which loop should be spatial/temporal?
 - Data/Model Parallelism

/ Review: Navigating the Mapping Space



- DNN mapping problem can be viewed as an optimization problem.
- Given:
 - DNN dimensions (N, H, W, C, R, S, K, stride, padding)
 - Hardware specifications (dataflow, memory hierarchy)
- Find:
 - An optimal loop nest that minimizes latency and/or energy.
 - That defines both temporal and spatial execution order
- Approaches:
 - Exhaustive search
 - Random search
 - Learning-based algorithms

Review: 软件栈（以华为昇腾为例）



四个层次，一个辅助工具链。

L3应用使能层：应用级封装，面向应用领域提供处理算法。

L2执行框架层：将算法进行深度学习框架的调用（框架管理器），离线深度学习模型生成（流程编排器）。

L1芯片使能层：给硬件通过加速库输出算子层面的任务，类似于生成机器码。

L0计算资源层：实际硬件。

工具链：开发平台。

➤ **A Detailed Look at Toolchain**

- **Concept**

- **Optimization & Case Study**

➤ **Data Orchestration**

- **Design Principles**

- **Implicit vs Explicit**

- **Coupled vs Decoupled**

➤ **Memory Subsystem Design**

- **Memory metrics**

- **Memory Technologies**

/Toolchain

- 昇腾/TVM等等



工具链帮助从
算法到硬件的
快速部署。

IR: Intermediate
Representation

- 为什么我们需要工具链？
 - 工具链能根据不同的硬件平台，进行差异化的部署策略。
- Case:



(a) NCHW



(b) NHWC

存储Activation时，采用RRRGGBBBB的排布还是RGB-RGB-RGB的排布顺序？

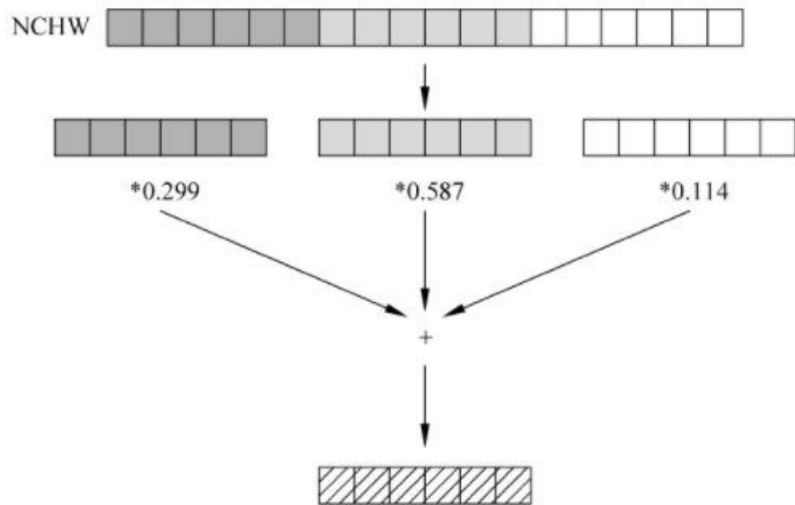


图 5-12 NCHW 实现 RGB 转灰度值计算^①

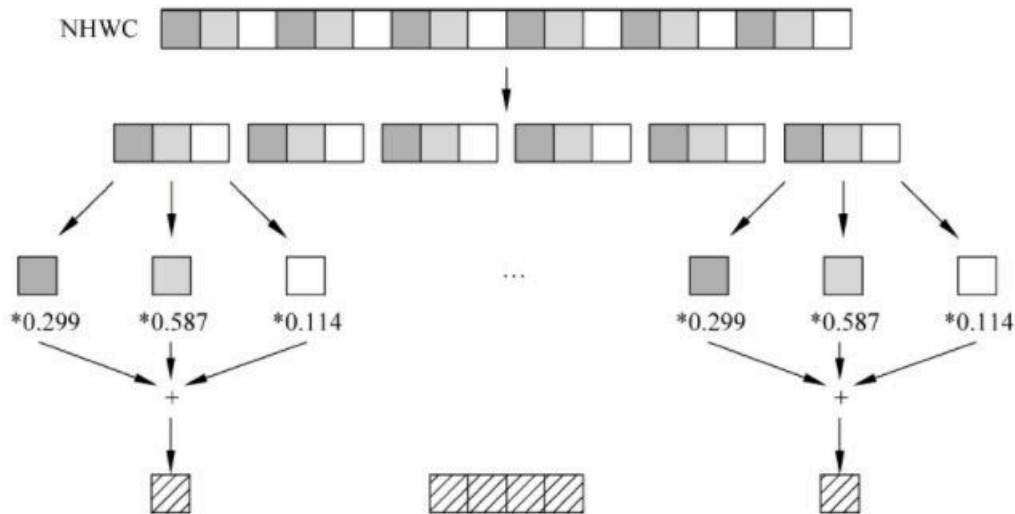
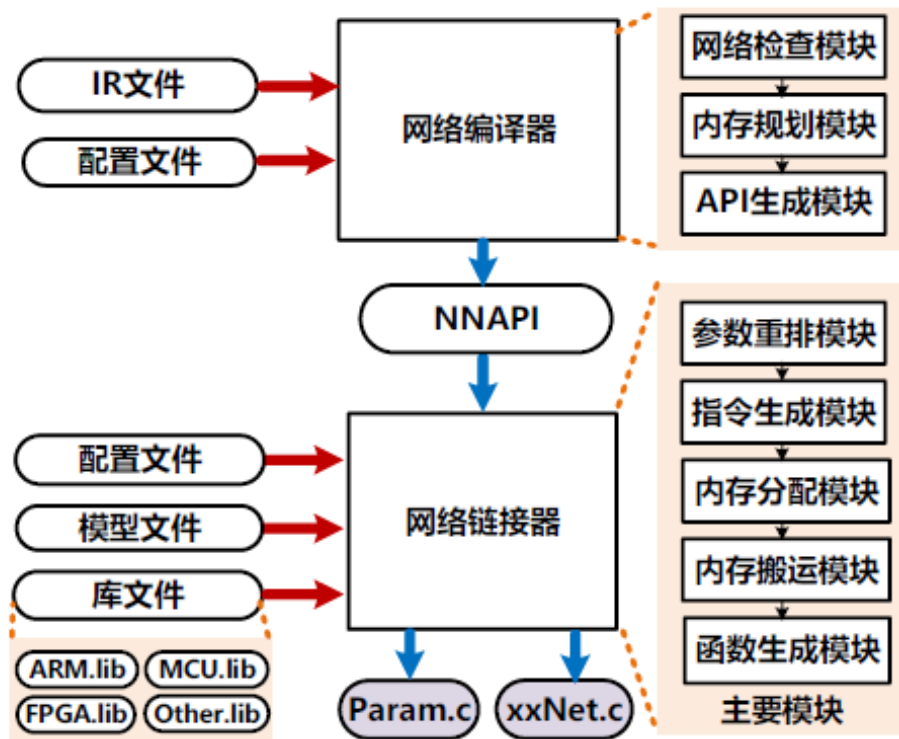


图 5-13 NHWC 实现 RGB 转灰度值计算^①

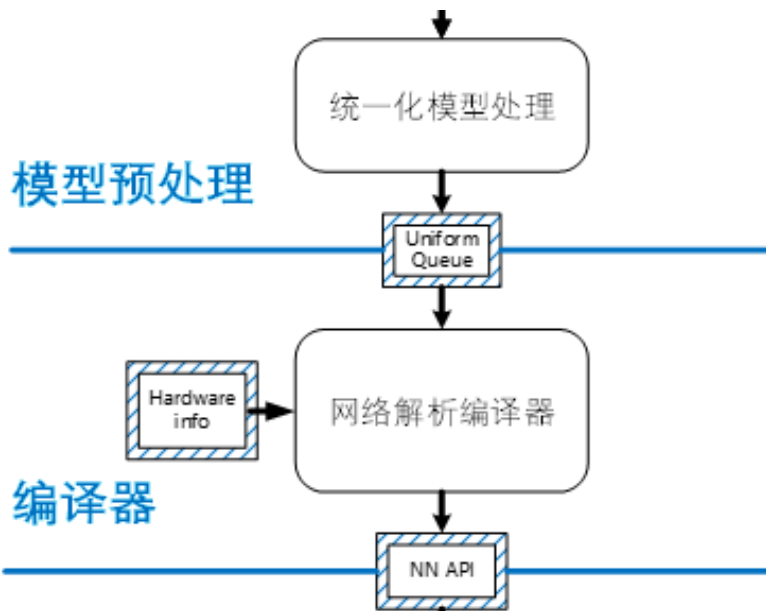
- 计算规律强，控制简单，但需要较大瞬时内存带宽和临时空间
- 可以边读边算（内存带宽需求小），临时空间需求小，控制复杂
- 工具链可以根据硬件和任务信息，来选择不同的部署方式

案例：Sticker工具链



- 网络编译器用于预处理文件和顶层部署优化
- 网络链接器用于划分计算方式以及连接库文件。

案例：Sticker-网络编译器



- 输入
 - 网络定义及参数信息 (IR)
 - 硬件信息 (型号, 厂家、存储资源, 是否有外接存储设备, 外部存储空间首地址等)
- 根据输入, 自动分配存储空间, 产生计算顺序并优化。
- 输出
 - NN API

案例：Sticker的IR文件



```
{
  "name": "FeatureExtractor_MobilenetV2_expanded_conv_depthwise_depthwise_Fold",
  "operation": "dwconv",
  "activation_type": "Relu6",
  "input_log2scale": 4,
  "weight_log2scale": 4,
  "bias_log2scale": 4,
  "output_log2scale": 4,
  "output_shift": 4,
  "bias_shift": 4,
  "load_bias": true,
  "input_channel_num": 16,
  "output_channel_num": 16,
  "input_size": {
    "height": 135,
    "width": 240
  },
  "padding": {
    "top": 1,
    "bottom": 1,
    "left": 1,
    "right": 1
  },
  "stride": {
    "height": 1,
    "width": 1
  },
  "kernel_size": {
    "height": 3,
    "width": 3
  },
  "output_size": {
    "height": 135,
    "width": 240
  },
  "input_dtype": "int8",
  "output_dtype": "int8",
  "weight_dtype": "int8",
  "bias_dtype": "int8",
  "previous_layer": [
    "FeatureExtractor_MobilenetV2_Conv_Conv2D_Fold"
  ],
  "next_layer": [
    "FeatureExtractor_MobilenetV2_expanded_conv_project_Conv2D_Fold"
  ]
}
```

- IR文件本质是对一层信息的收集
- 每一个计算节点，都会包含该层的所有所需信息

/ TVM工具链的优化策略



```
OPT_PASS_LEVEL = {  
    "SimplifyInference": 0,  
    "OpFusion": 1,  
    "FoldConstant": 2,  
    "CombineParallelConv2D": 3,  
    "FoldScaleAxis": 3,  
    "AlterOpLayout": 3,  
    "CanonicalizeOps": 3,  
    "EliminateCommonSubexpr": 3,  
}
```

代码 5-18 TVM Relay 提供的图优化方法

SimplifyInference: 简单推理, 不做任何优化。

OpFusion: 算子融合。

FoldConstant: 常量折叠。

CombineParallelConv2D: 结合并行的卷积与运算。

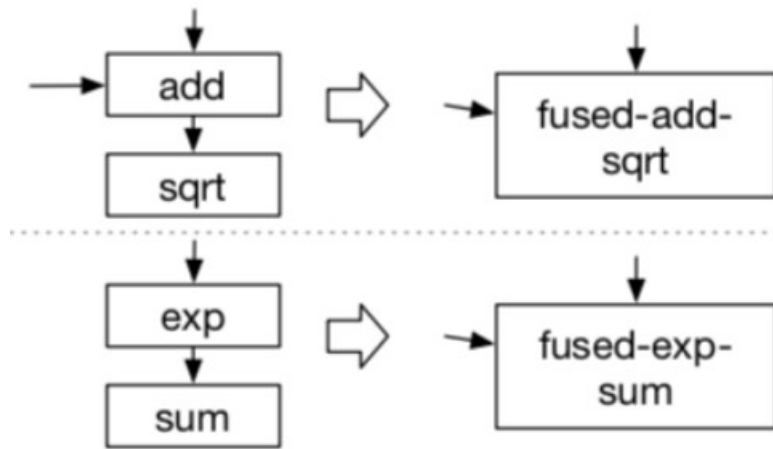
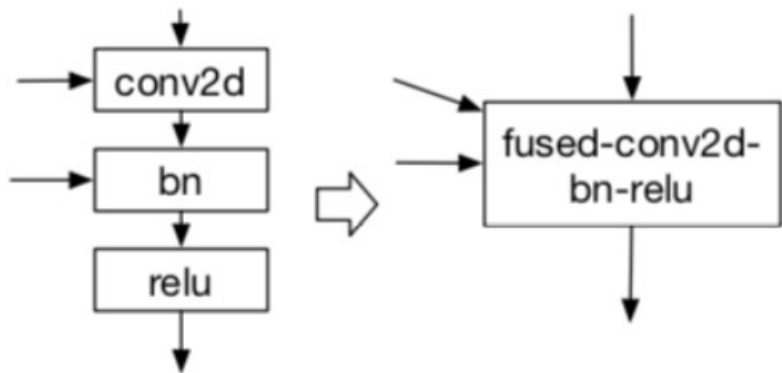
FoldScaleAxis: 折叠缩放轴。

AlterOpLayout: 改变算子排布。

CanonicalizeOps: 规范化算子。

EliminateCommonSubexpr: 消除公共子表达式。

/ 算子融合

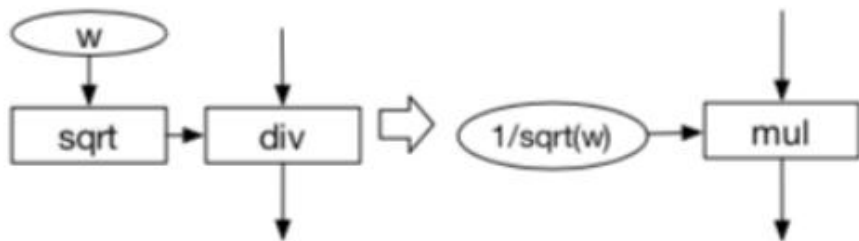


- 融合算子，减少指令数，避免来回访存。
- 简单数学操作融合。

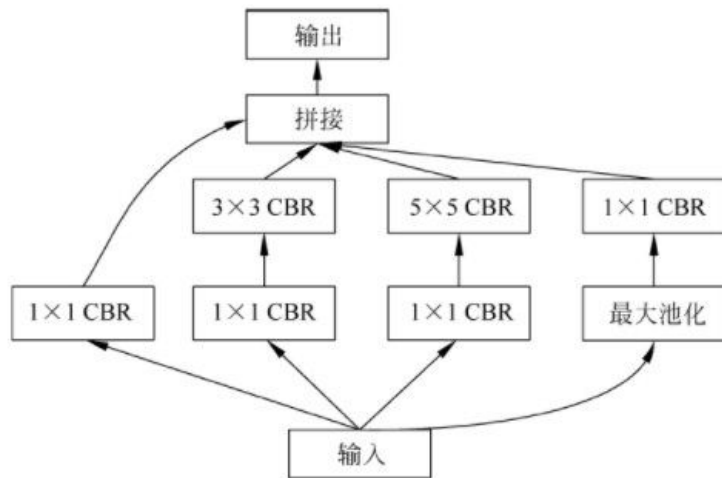
常量折叠与并行卷积



Constant Parameter Path Pre-Computation
(w is marked as a constant parameter)

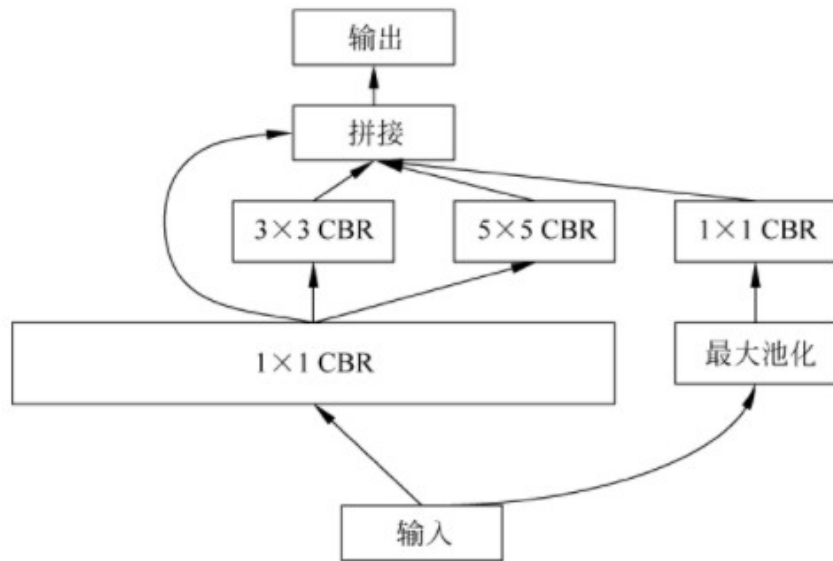
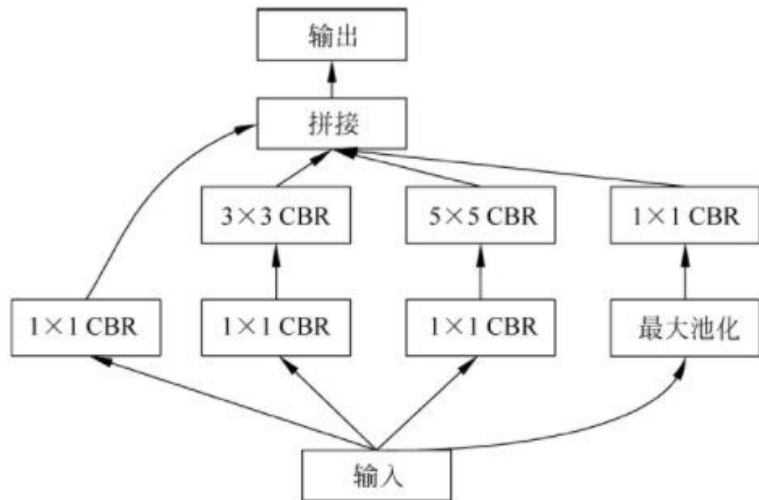


- 在编译时直接得到常量



- 计算资源足够时，进行模块间的并行运算

存储复用



- 三个 1×1 模块的输入存储可以复用

Mapping中的切片优化

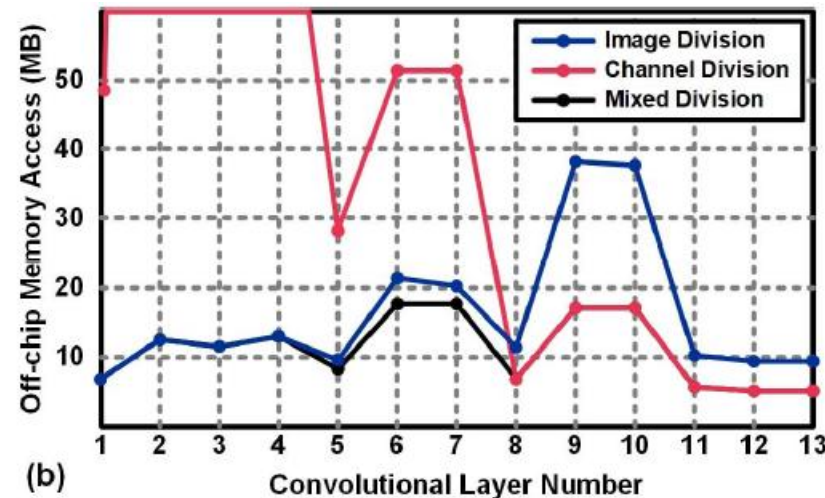


	Off-chip Access (W/O Compression Scheme)					
	Image Division		Channel Division		Mixed Division	
Input Image	$W_i \times H_i \times C_i$		$W_i \times H_i \times C_i$		$W_i \times H_i \times C_i$	
Output Image	$W_o \times H_o \times C_o$ Pooling Size		$W_o \times H_o \times C_o$ $\times (\text{Ch. Div. \#} \times 2)$		$W_o \times H_o \times C_o$ $\times (\text{Ch. Div. \#} \times 2)$	
Weight	$W_f \times H_f \times C_i \times C_o$ $\times \text{Img. Div. \#}$		$W_f \times H_f \times C_i \times C_o$		$W_f \times H_f \times C_i \times C_o$ $\times \text{Img. Div. \#}$	

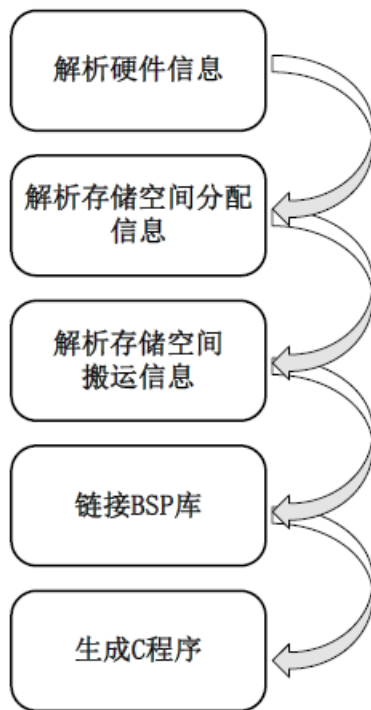
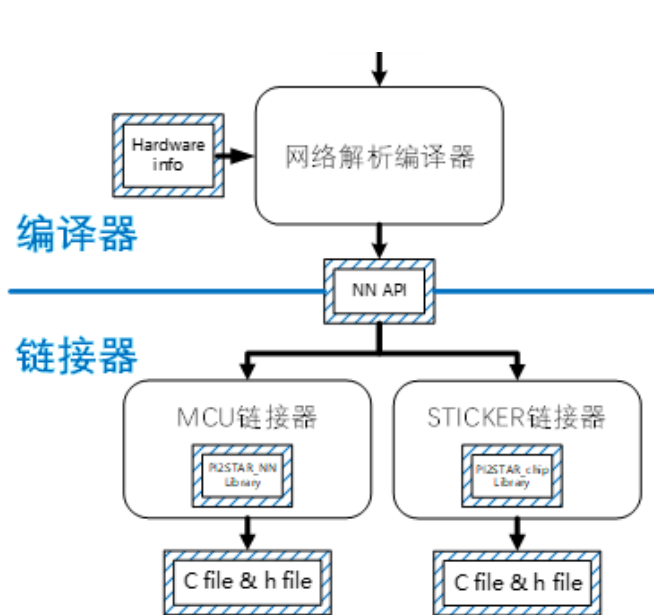
Img. Div. #	16	8	4	2	1
Ch. Div. #	1	2	4	8	16
Input Image	1.5	1.5	1.5	1.5	1.5
Output Image	1.5	6.1	12.3	24.5	49
Weight	18	9.0	4.5	2.3	1.1
Total (MB)	21.0	16.6	18.3	28.3	51.6

VGG-16 Example – 6th Convolution Layer

(a) $(W_i, H_i, C_i) = (56, 56, 256)$, $(W_o, H_o, C_o) = (56, 56, 256)$, $(W_f, H_f) = (3, 3)$

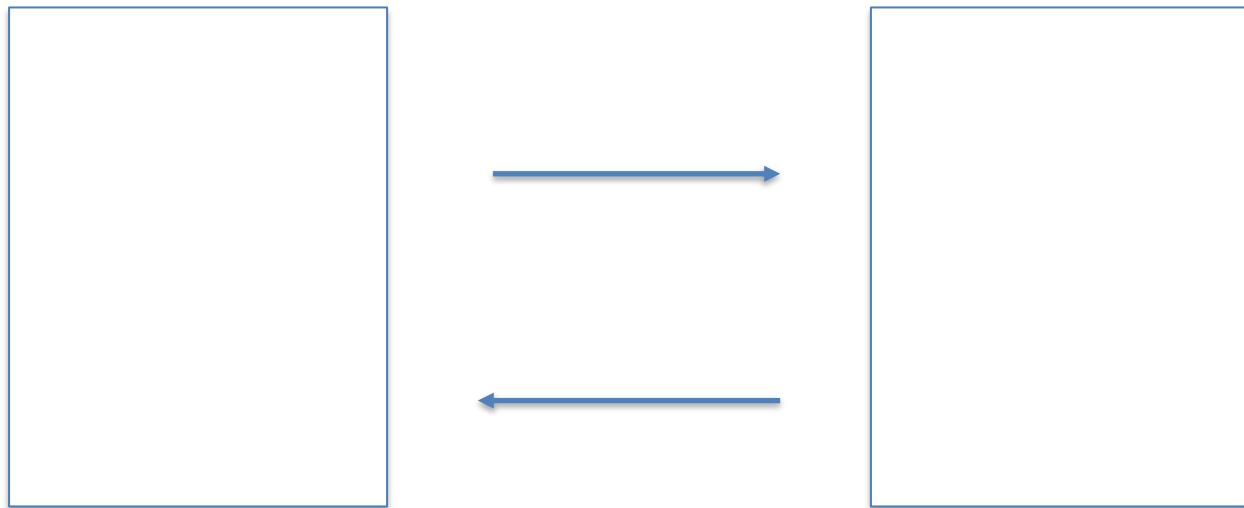


案例：Sticker-网络链接器



- 解析硬件信息API
- 解析存储空间分配API
- 解析存储空间搬运API
- 链接BSP库
- 生成C程序

/ 案例：Sticker



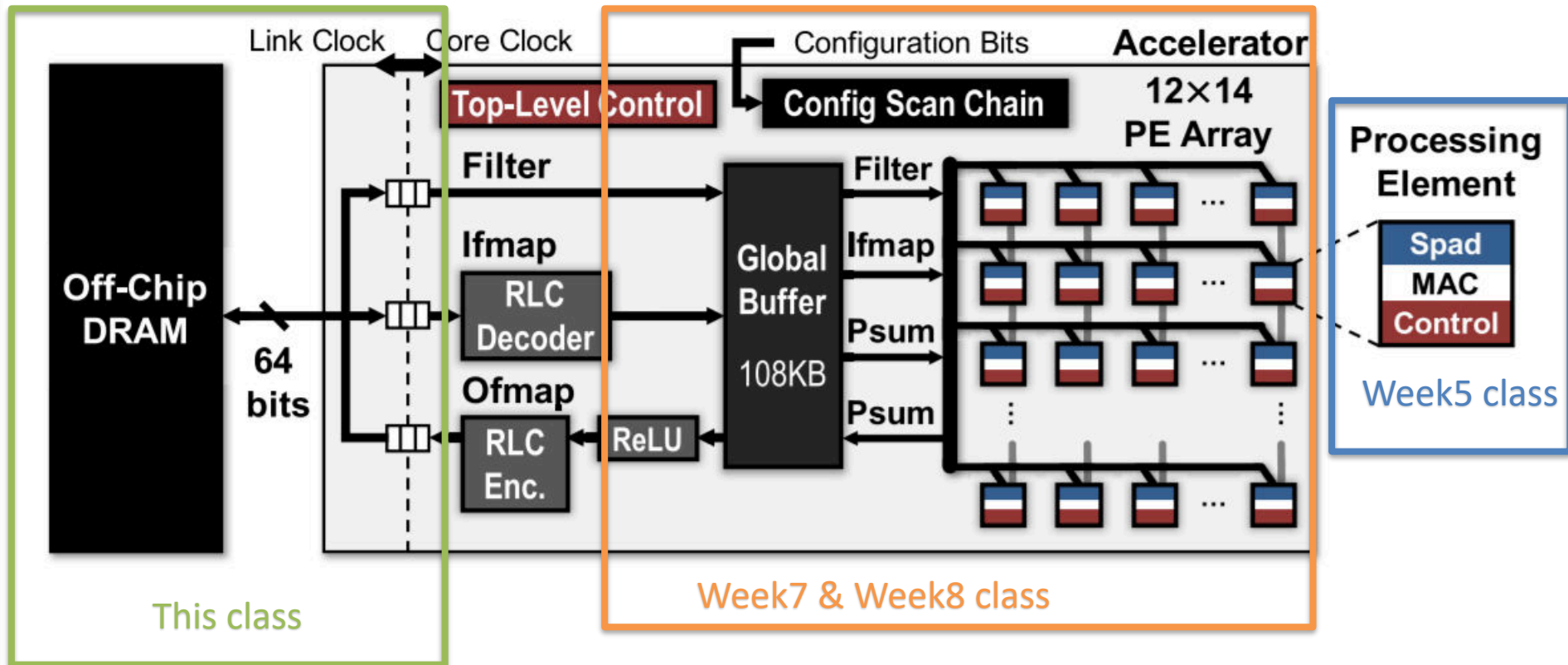
FPGA作为协处理器：加速神经网络

CPU作为主处理器：调度和控制

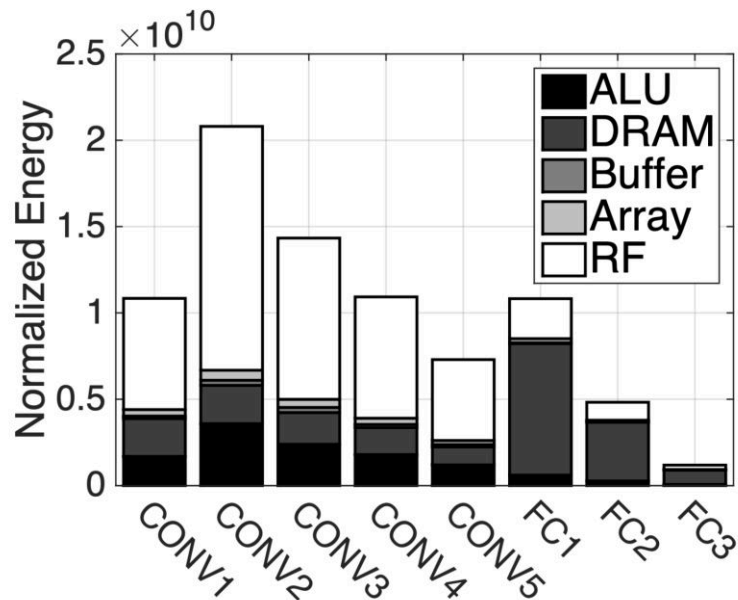
/ Overview

- **A Detailed Look at Toolchain**
 - **Sticker**
 - **Optimization & Case Study**
- **Data Orchestration**
 - **Design Principles**
 - **Implicit vs Explicit**
 - **Coupled vs Decoupled**
- **Memory Subsystem Design**
 - **Memory metrics**
 - **Memory Technologies**

/ A Typical AI Accelerator



存储系统对于AI计算的重要性



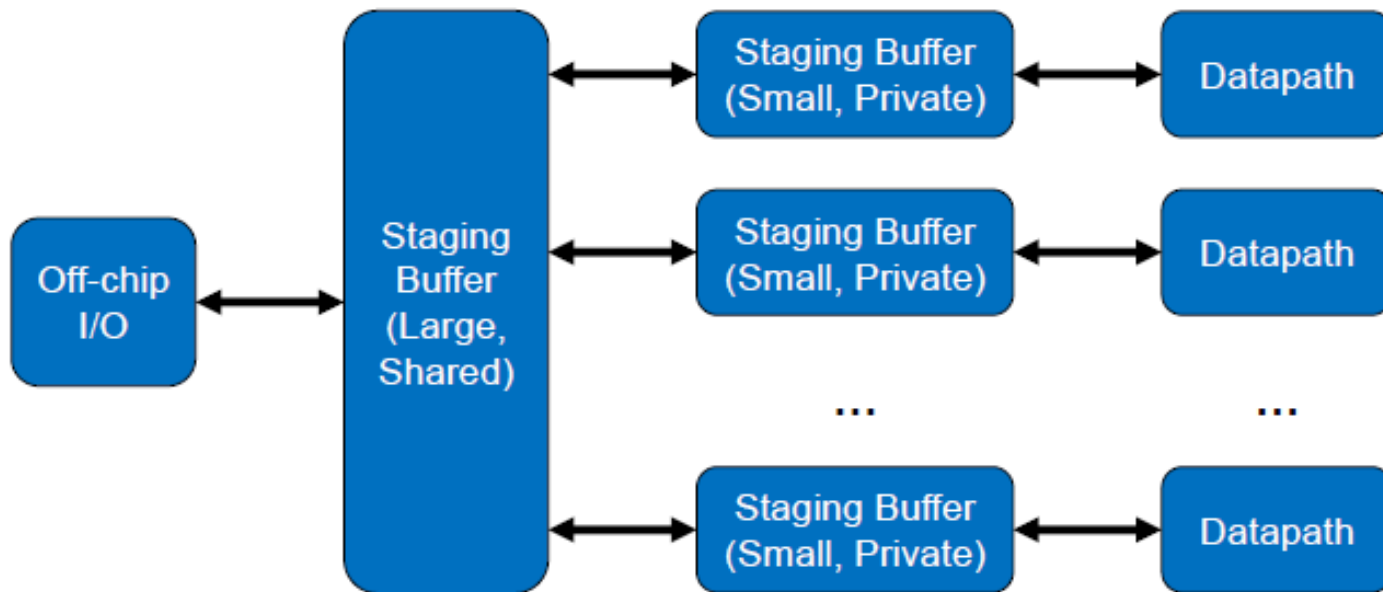
On-chip Buffer 芯片面积占比

DaDianNao [5]:	48%	Eyeriss [6]:	40%-93%
EIE [18]:	93%	SCNN [35]:	57%
TPU [22]	35%	PuDianNao [27]	63%

/What is data orchestration (数据编排)



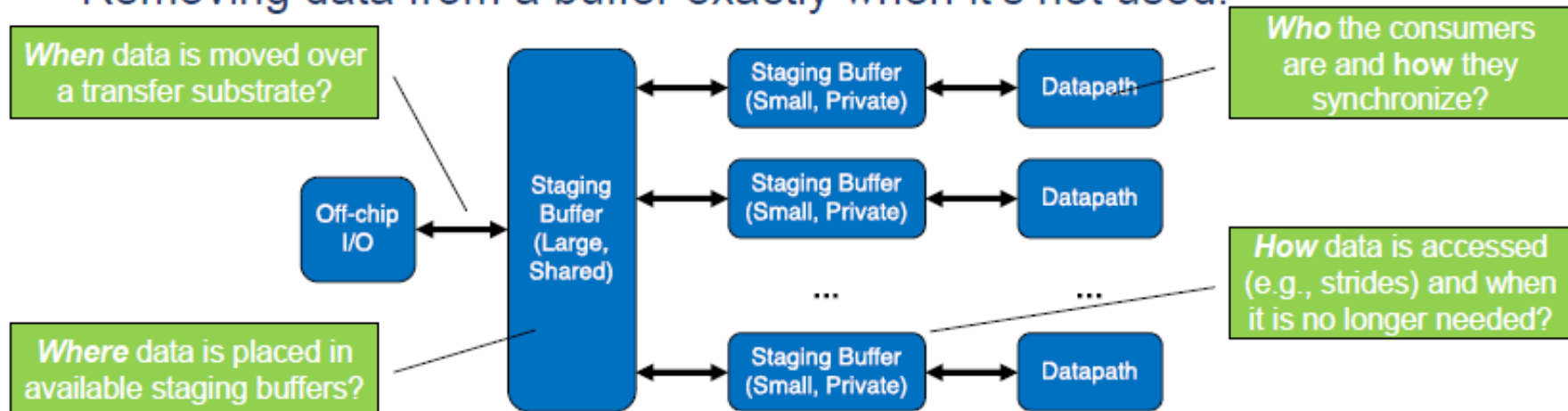
- Feeding data to a functional unit exactly when it wants it.
- Removing data from a buffer exactly when it's not used.



What is data orchestration



- Feeding data to a functional unit exactly when it wants it.
- Removing data from a buffer exactly when it's not used.



- ML accelerators use workload knowledge to optimize data orchestration at design time.

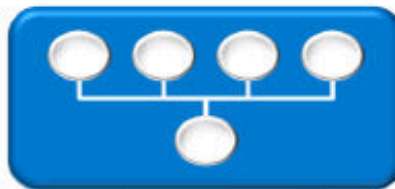
/ Design Principles



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University



Local reuse - staged physically close to consuming units



Cross-unit use - amortize data access and communication



Bandwidth efficiency - Maximize delivery rate by controlling outstanding requests



Delivery/use overlap - Next tile should be available when current is done (e.g., double-buffering)



Precise synchronization - Only wait for exactly data you need, respond quickly (e.g., no barriers or remote polling)



Simple structures - Minimize hardware area/power

Buflets, ASPLOS 2019

/ Agents in Data Orchestration

- Data Producer:
 - The agent that currently contains the requested data.
- Data Consumer
 - The agent that consumes the requested data.
- Data Requestor
 - The agent that sends out data requests.
- Data Distributor
 - The agent that distributes data requests.

Producer

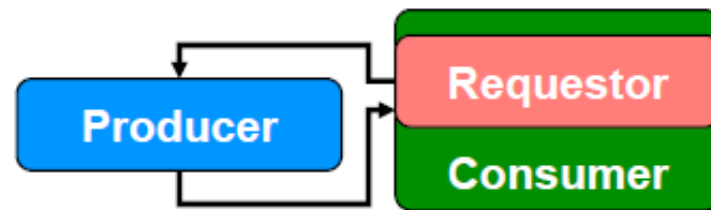
Consumer

Requestor

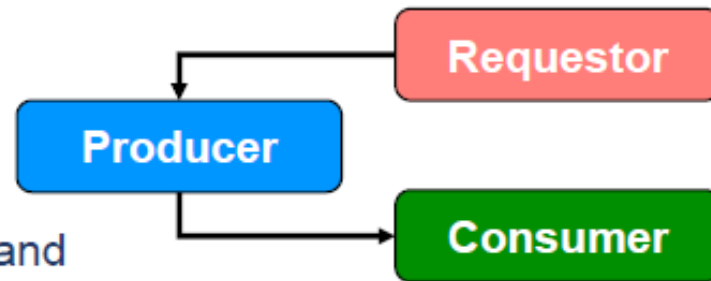
Distributor

/ Coupled vs Decoupled

- Whether requestor == consumer.
- Coupled
 - Requestor is the same as Consumer.
 - Pro: Easy and intuitive synchronization.
 - Con: Reserved landing zone (e.g., registers) for incoming data.
- Decoupled
 - Requestor is different from Consumer.
 - Pro: Requestor can run ahead.
 - Con: Synchronization between requestor and consumer.



Coupled

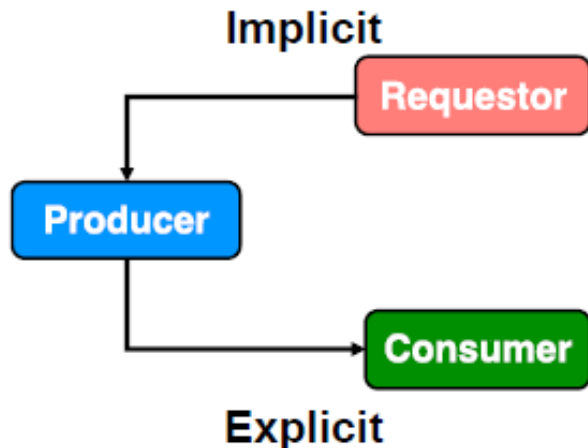
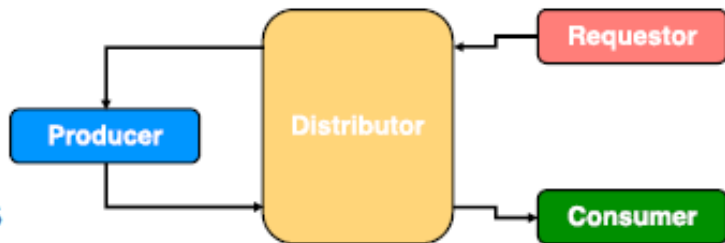


Decoupled

/ Implicit vs Explicit



- Whether requests are sent to data producer directly.
- Implicit:
 - Requestor is not aware of where the data is and when the data is evicted.
 - Pro: Easy to program; Workload agonistic.
 - Con: Area and energy overhead for tags etc.
- Explicit:
 - Requestor explicitly interacts with producer.
 - Pro: Low overhead.
 - Con: Hard to program.



/ Data Orchestration Taxonomy

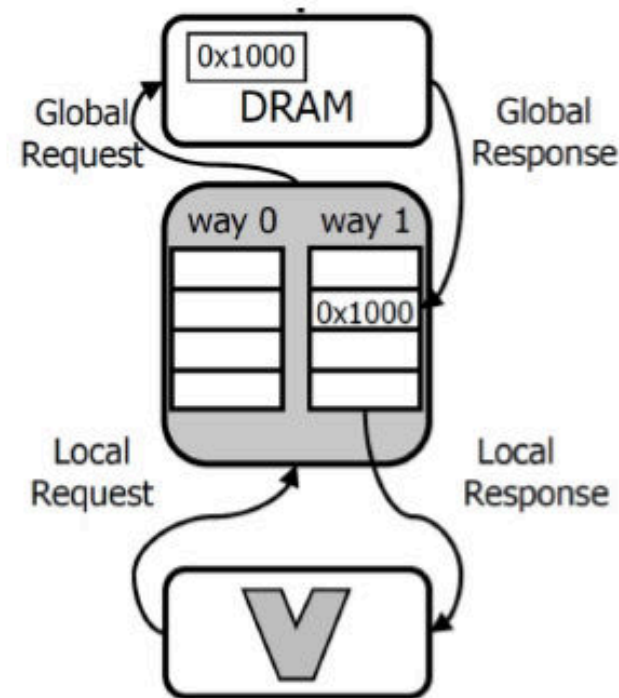


	Coupled	Decoupled
Implicit	Cache	Decoupled Access-Execute
Explicit	GPU shared memory	DMA

/ Implicit-Coupled Data Orchestration



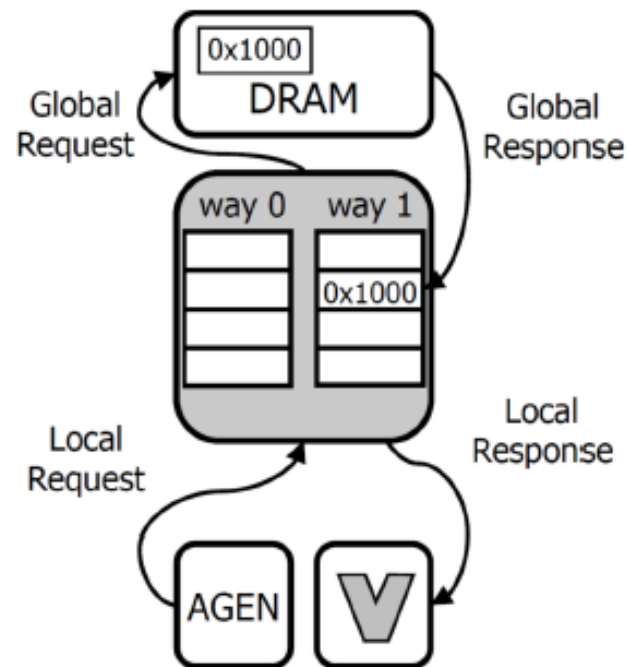
- Cache
 - Widely used in general-purpose computing
 - Reusable, composable
 - High area and energy overhead
- Implicit:
 - Requestor, i.e., core, sends requests to L1 without knowing the exact location of data.
- Coupled:
 - Unified pipeline for mem and compute instructions.



/ Implicit-Decoupled Data Orchestration



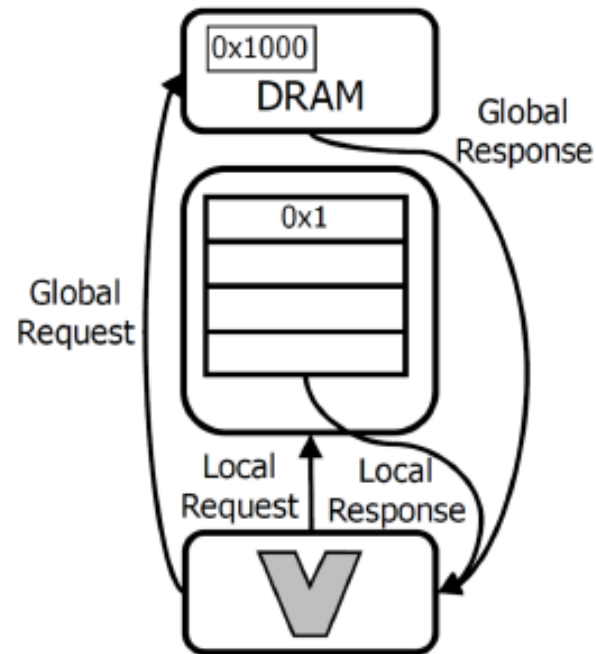
- Decoupled Access-Execute
 - Separating data requestor and consumer connected via hardware queues.
 - Targeting general-purpose computing.
 - Allowing both to proceed at their own rates.
- Implicit:
 - Requestor, or accessor, sends requests to L1 without knowing the exact location of data.
- Decoupled:
 - Separate requestor/accessor and consumer/executor



/ Explicit-Coupled Data Orchestration



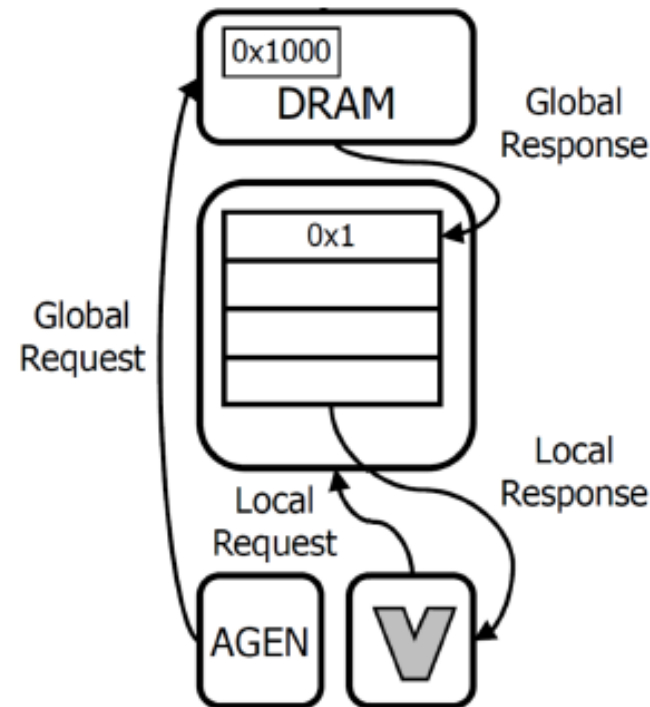
- GPU Shared Memory
 - Scratchpad in GPU
 - Instructions to explicitly move data across memory hierarchy
- Explicit:
 - Core explicitly requests data from DRAM
- Coupled:
 - The same pipeline for data requests and consumption.



/ Explicit-Decoupled Data Orchestration



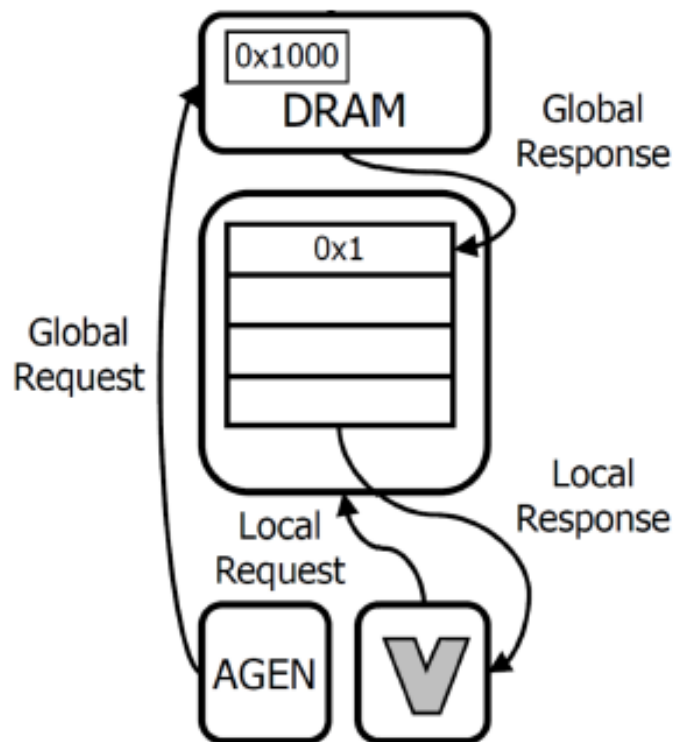
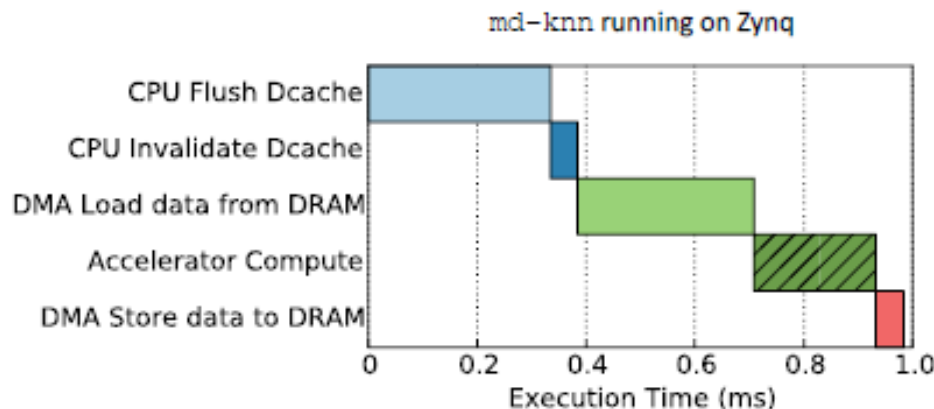
- EDDO
- DMA engines
 - Direct-Memory Access
 - Acts as an address (request) generator to DRAM
 - “Pushing” data to consumer
- Explicit:
 - Software-managed data placement
- Decoupled:
 - DMA only does data orchestration not compute.



/EDDO for Domain-Specific Accelerators



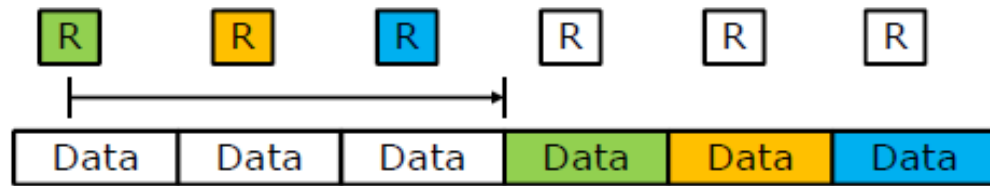
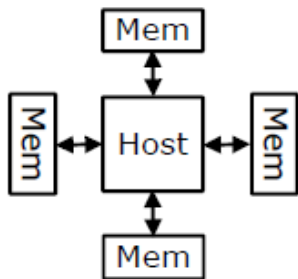
- Benefits:
 - Leverage static workload knowledge
 - Efficient hardware implementations
- Challenges:
 - Synchronization in a decoupled system



/ Overview

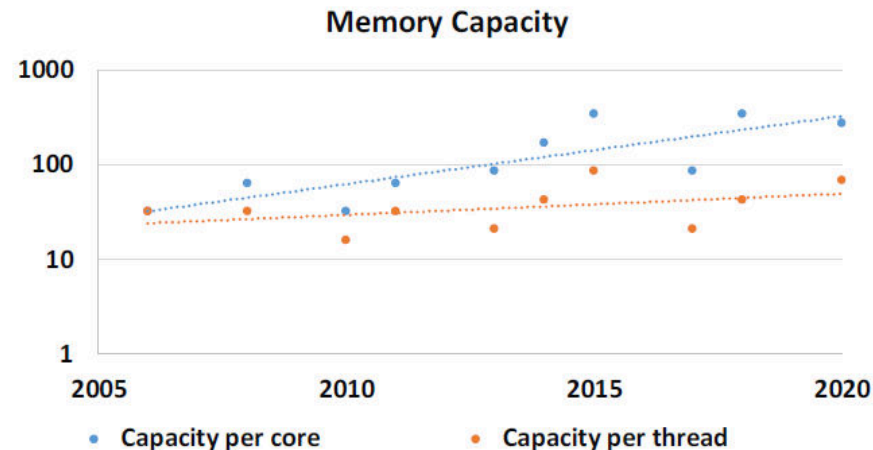
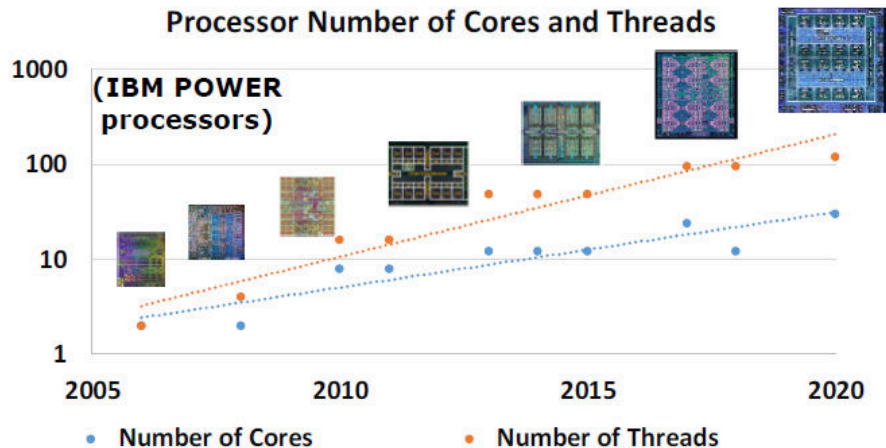
- **A Detailed Look at Toolchain**
 - **Sticker**
 - **Optimization & Case Study**
- **Data Orchestration**
 - **Design Principles**
 - **Implicit vs Explicit**
 - **Coupled vs Decoupled**
- **Memory Subsystem Design**
 - **Memory metrics**
 - **Memory Technologies**

/Memory metrics



- Bandwidth
 - Amount of data that memory can read or write per second (e.g. 100GB/s)
- Latency
 - Time delay from read request to data

Memory metrics



- Capacity
 - Memory
 - Number of processor cores and threads
- Power

/Cost



- Cost / bit [\$/GB] (as of Oct. 2020)
 - DRAM ~ \$4.2/GB
 - NAND ~ \$0.11/GB
- Assume
 - Intel i9 (8 cores, 2 threads / core)
 - 32GB / thread (-> 512GB)
 - CPU cost = \$380
 - DRAM cost = \$2,150
- Memory may be the major cost contributor
 - Lower cost memories are available (e.g. NAND), but with much lower performance (BW & latency)
- Memory cost
 - 5 – 20% in client systems
 - 20 – 50% in server systems

/ Design variables



Latency,
BW,
reliability



DRAM

- DDRx (DDR4, DDR5, [DDR6])
- HBMx (HBM2/2E, HBM3, [HBM4])
- LPDDRx (LPDDR5/5X, [LPDDR6])
- GDDRx (GDDR6/6X, [GDDR7])

BW, capacity,
and cost tradeoff

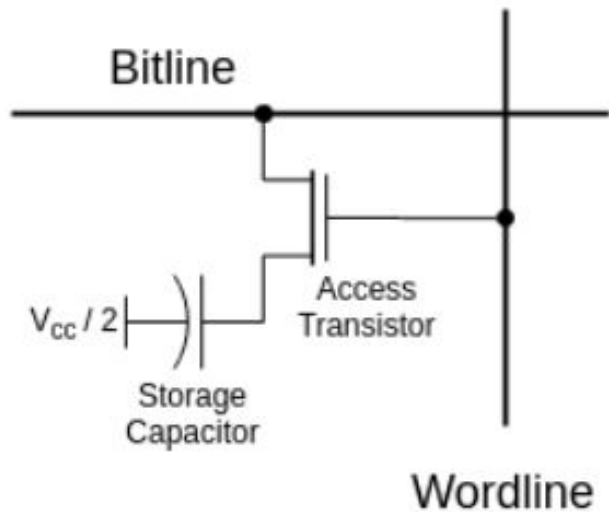
Capacity,
cost



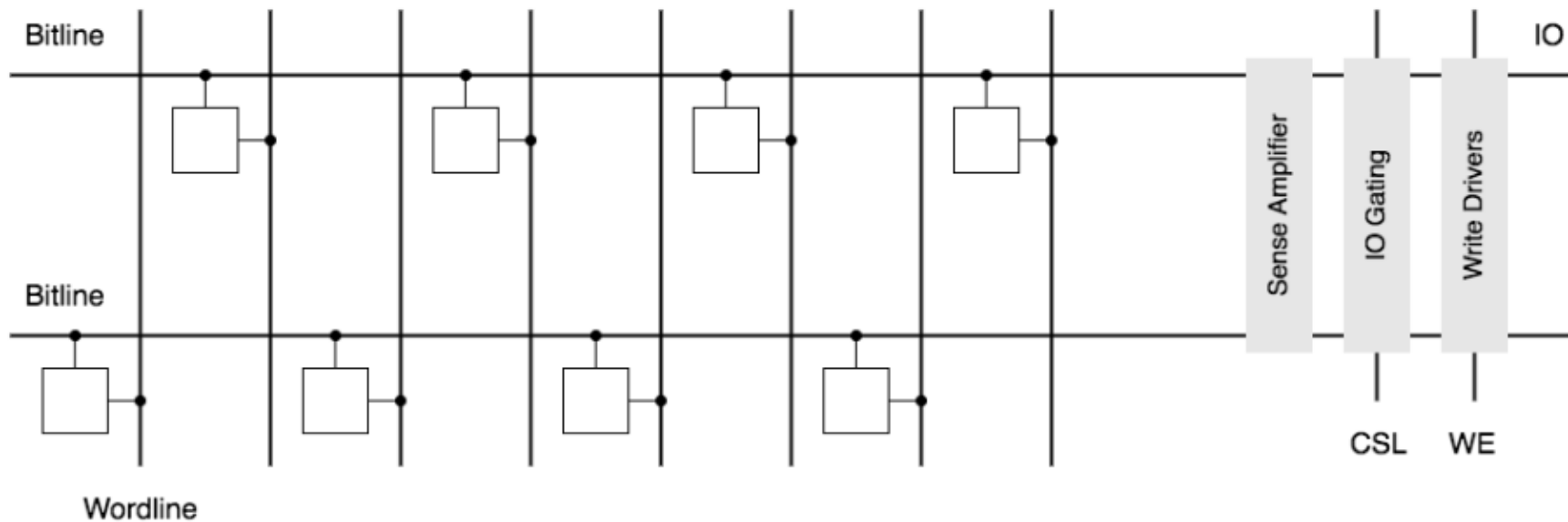
Non-Volatile Memory (NVM)

- NAND
- PCM (including 3DXP)
- RRAM
- MRAM
- Etc.

Capacity, latency



- 1bit存储电容
- 晶体管：控制对电容的读取和改写
- 字线：决定导通或截止
- 位线：外界经此访问电容上的数据



一个Wordline有效，会打开它控制的一列所有数据。

□ Standard DRAM families

■ Main memory (commodity) DRAMs

□ DDR/DDR2/DDR3/DDR4/DDR5

■ Mobile DRAMs

□ LPDDR/LPDDR1/LPDDR2/LPDDR3/LPDDR4/LPDDR5

□ WIO/WIO2

■ Graphics DRAMs

□ GDDR/GDDR2/GDDR3/GDDR4/GDDR5/GDDR6

■ HPC DRAMs

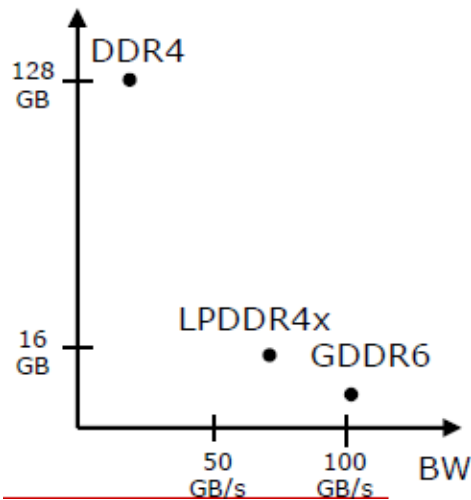
□ HBM/HBM2/HBM2E/HBM3

/DRAM



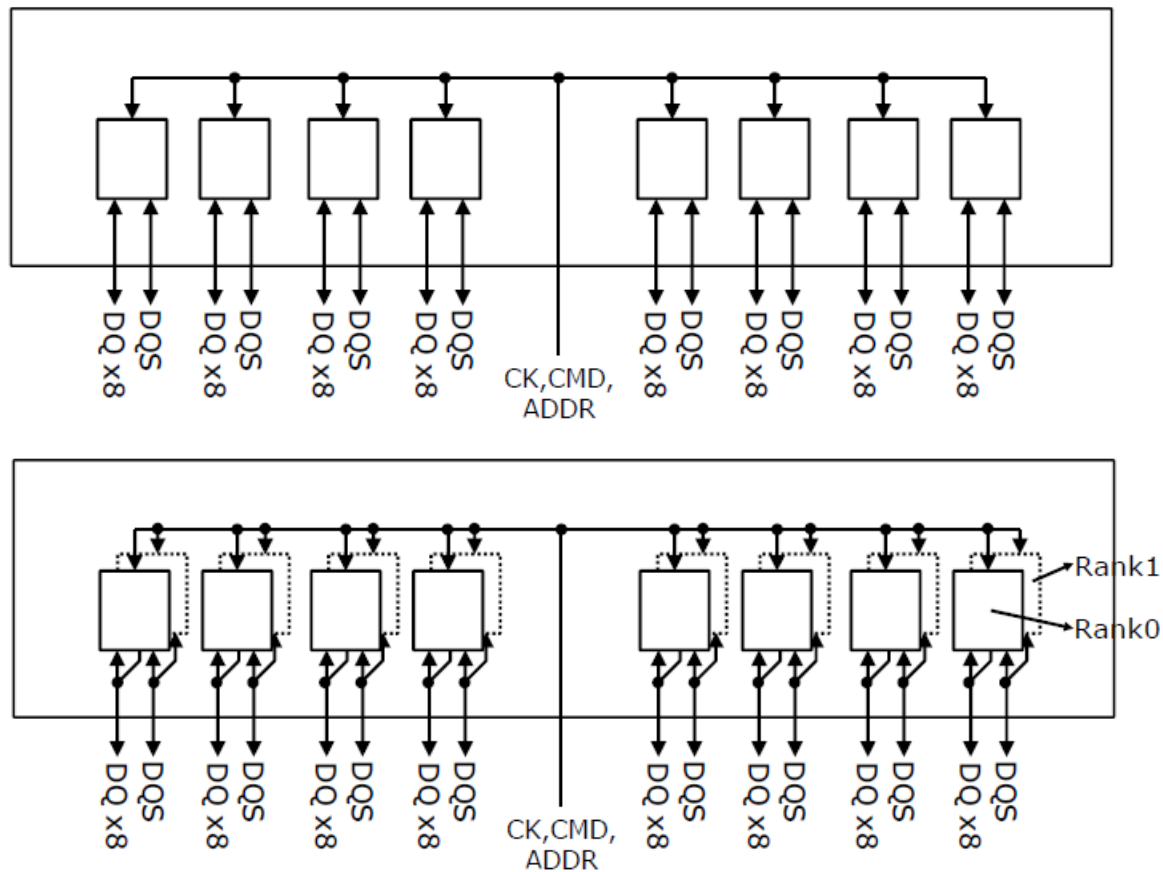
	DDR4	LPDDR4x	GDDR6
Per-pin speed	3.2Gbps	4.26Gbps	14Gbps
VDD	1.2V	1.1V	1.35V
Cost / bit	1x	~1.2x	~2x
Bus width per PKG	x4/x8/x16	x32/x64	x16/x32
BW per PKG	1.6-6.4GB/s	17-34GB/s	28-56GB/s
Max. capacity per PKG	8GB	8GB	2GB
BW per x64	25.6GB/s ¹⁾	68GB/s ²⁾	112GB/s ³⁾
Capacity per x64	128GB ¹⁾	16GB ²⁾	8GB ³⁾

Capacity

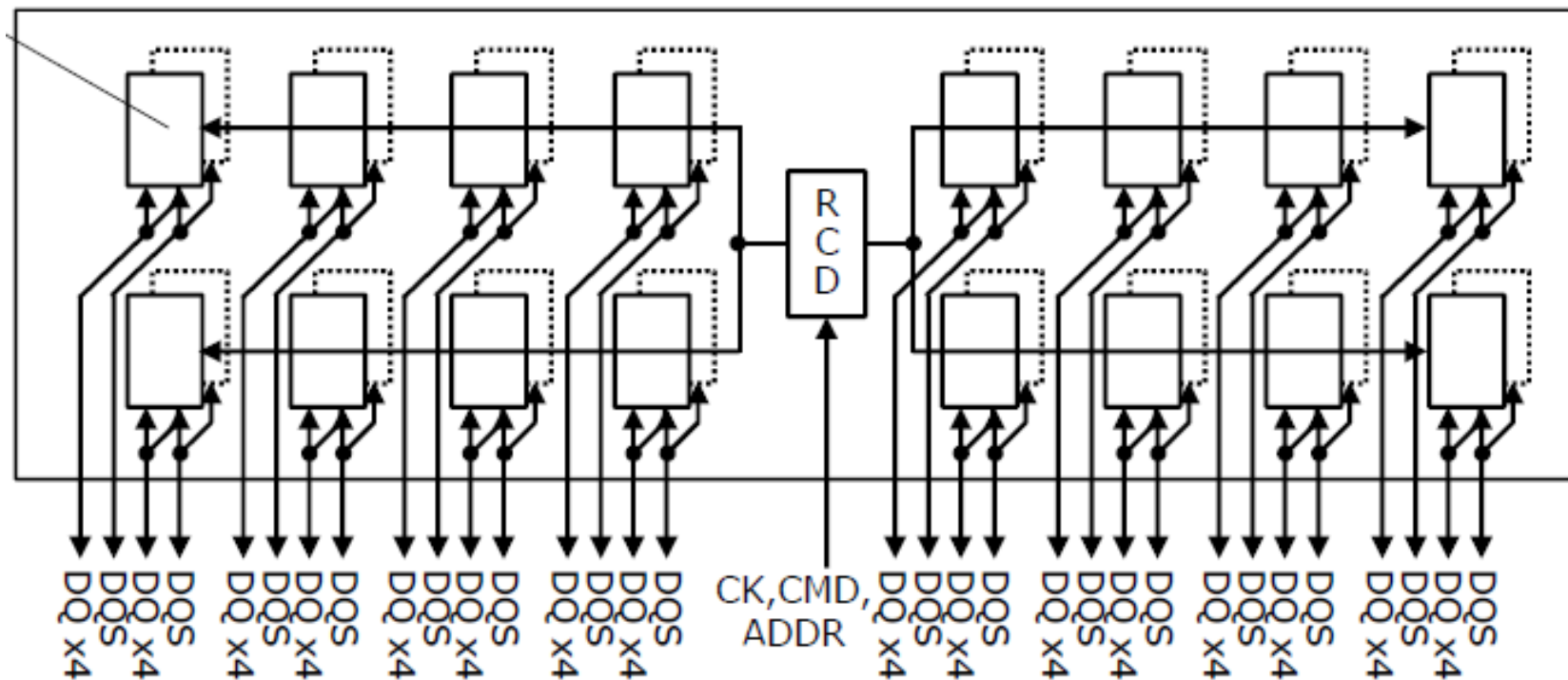


- LPDDR使用更低漏电的晶体管设计（但延时更长），用于手机等设备
- GDDR每一个pin脚传输速率是DDR的五倍，但它整体的面积更大，用于图形处理器

DIMM



/RDIMM

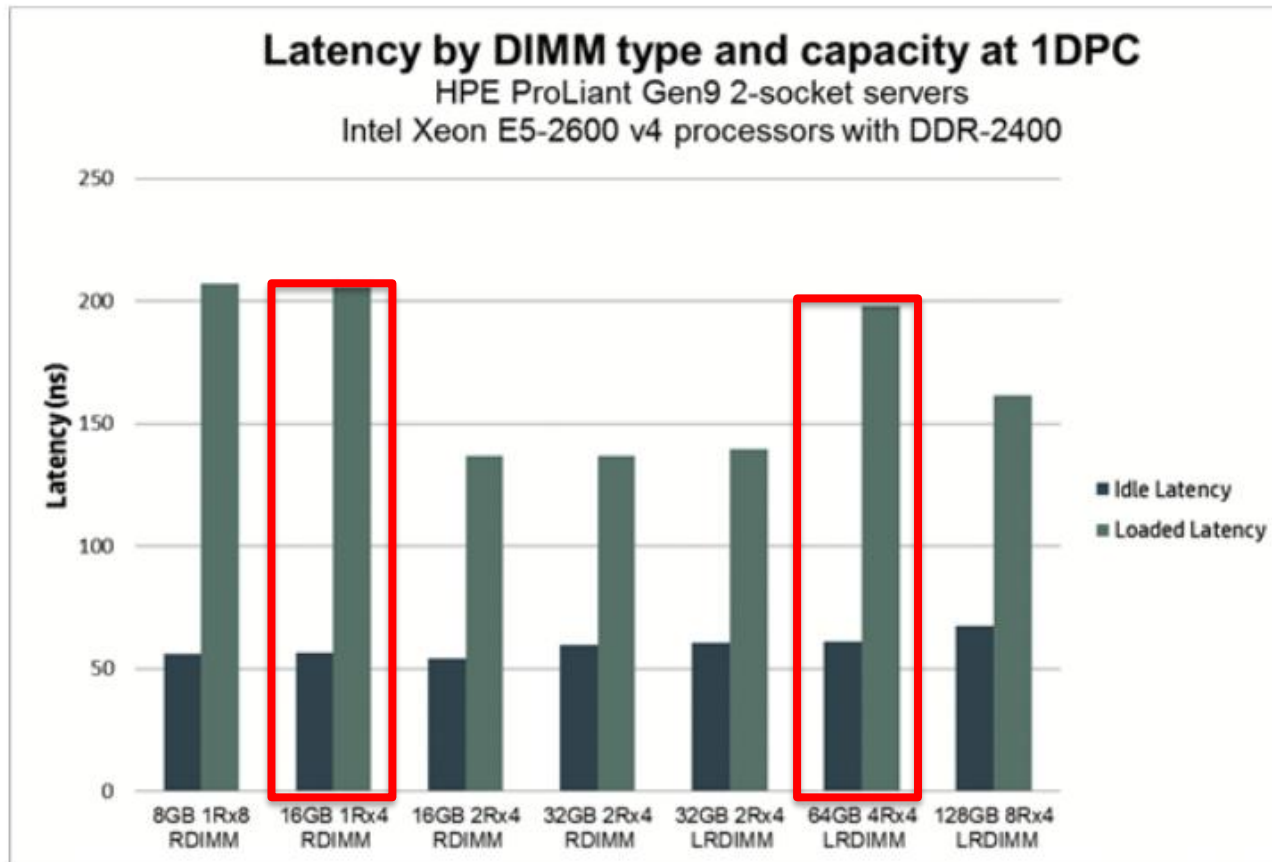


/DIMM-comparison

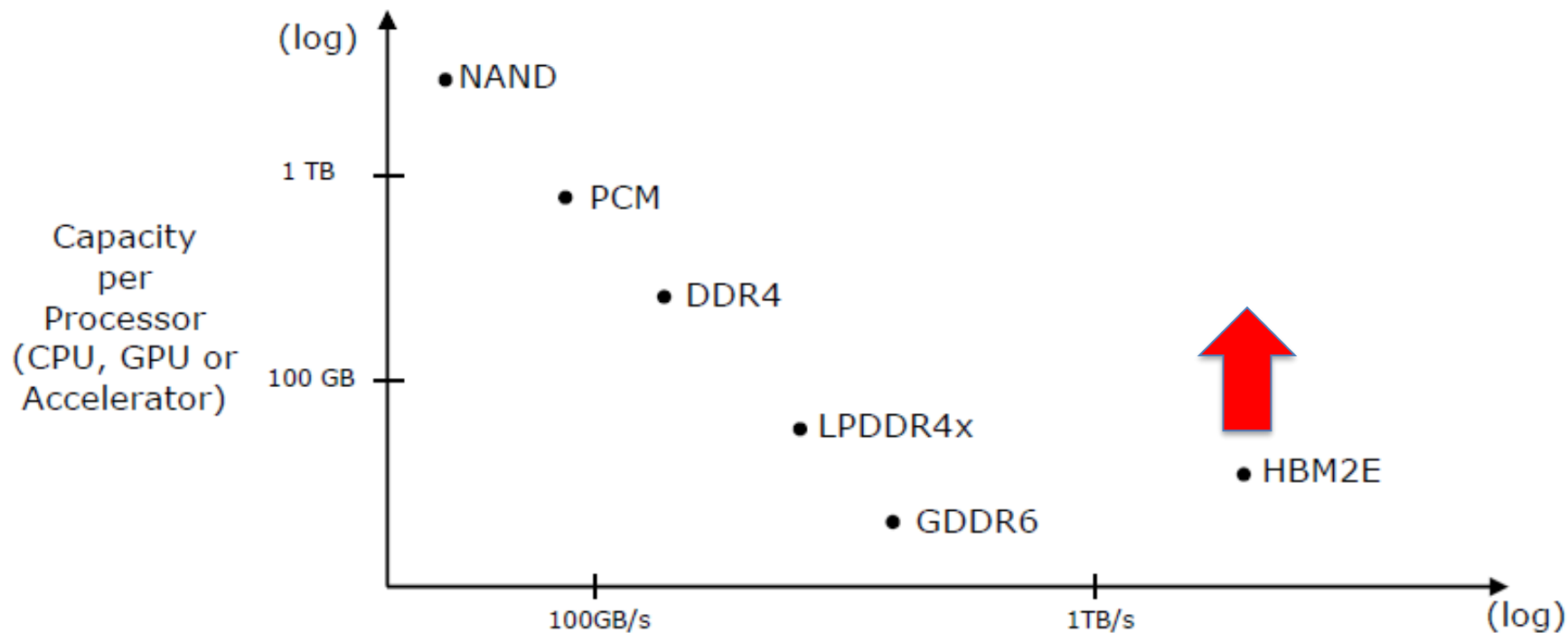


类型	技术	频率MT/S	容量	性能	价格	应用
UDIMM	DDR4 DDR3 DDR2 DDR SDRAM	266-2133	32MB- 4GB	低	低	桌面， 服务器
RDIMM	DDR4 DDR3 DDR2 DDR	333-3200	512MB- 32GB	较高	较高	服务器
LRDIMM	DDR4 DDR3	1333-3200	16GB- 64GB	高	高	服务器

/DIMM-comparison

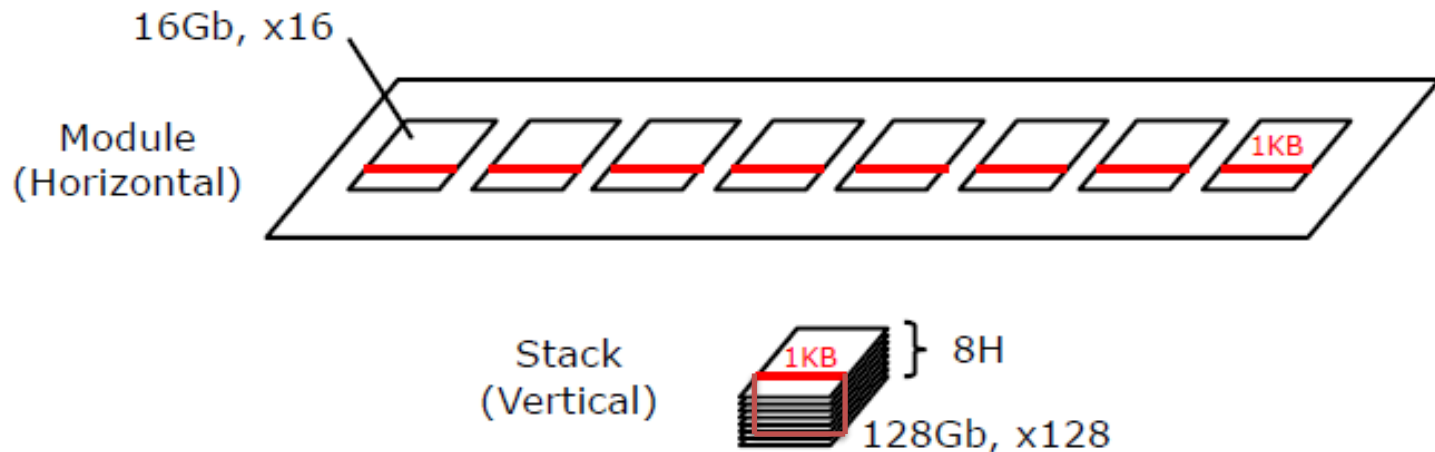


/ Comparison (DRAM & Non-DRAM)



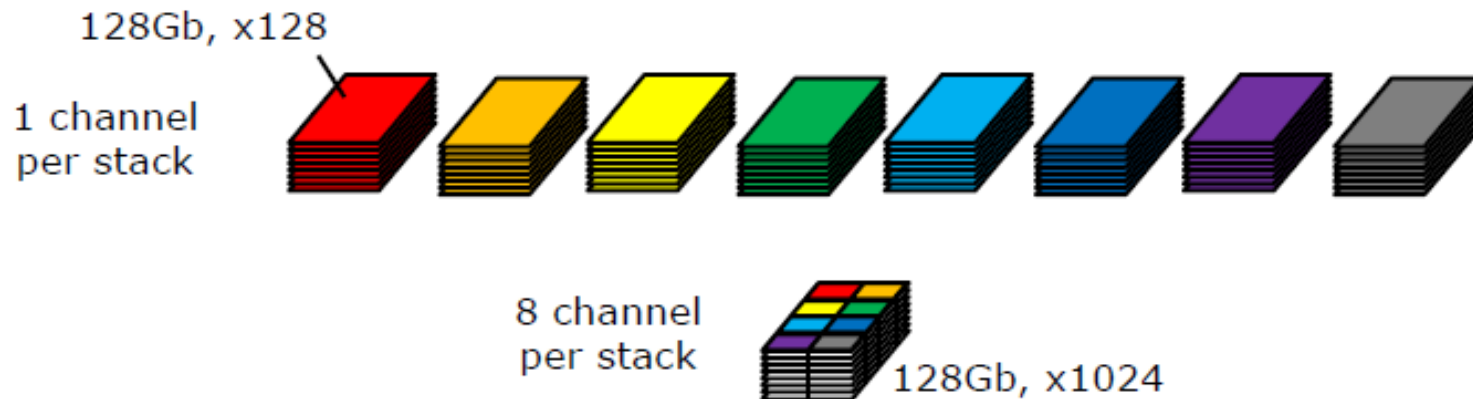
- HBM (High Bandwidth Memory)
- Discussion initiated to develop next generation graphics memory (post GDDR5)
- Two ways to achieve ultra high BW
 - Ultra high data rate
 - Ultra wide bus
- Positioned as the mainstream HPC memory due to wide usage of GPGPU in HPC

/ Deriving HBM

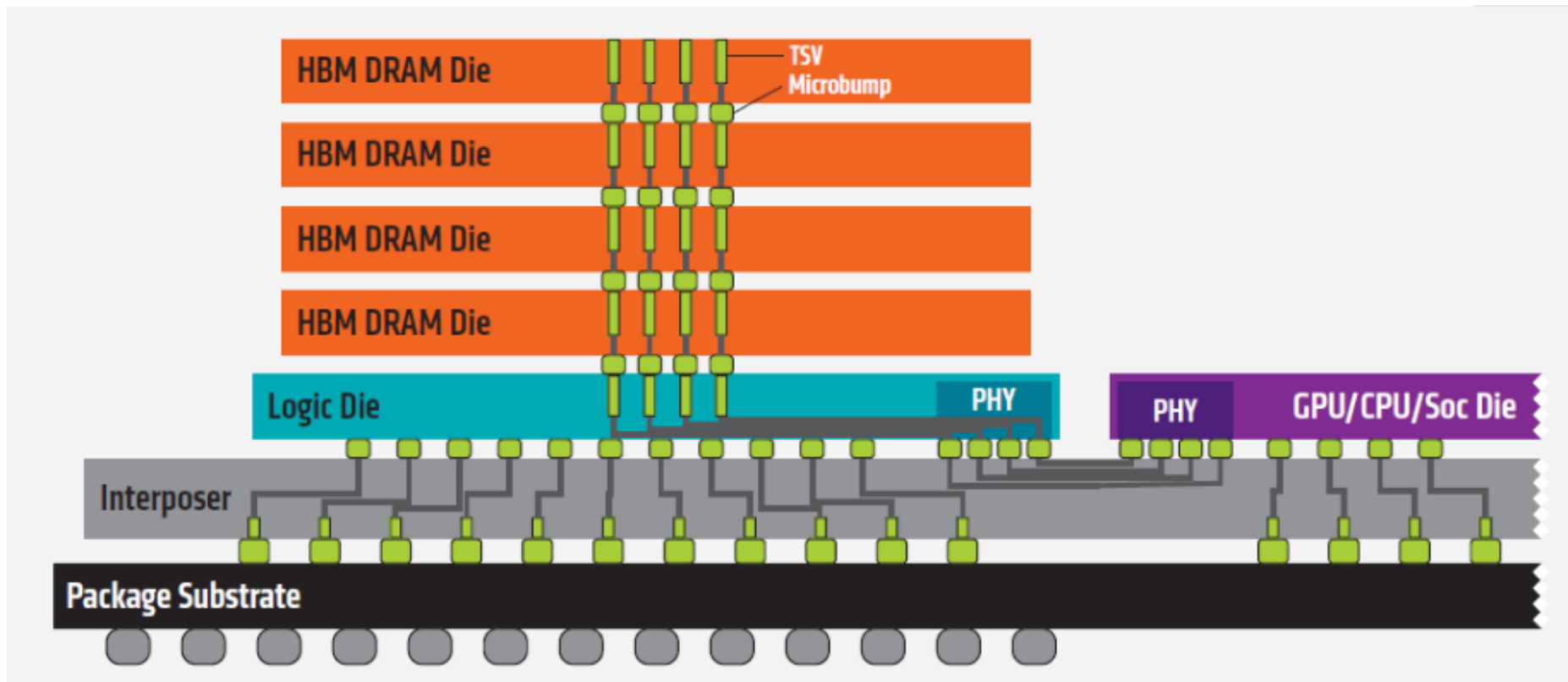


- ❑ 8x form factor saving
- ❑ 8x activation power saving
- ❑ 8x larger prefetch (internal data lines) [chip size overhead]

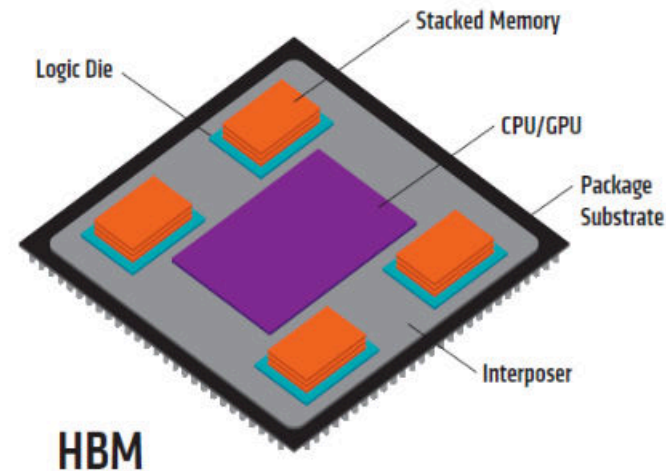
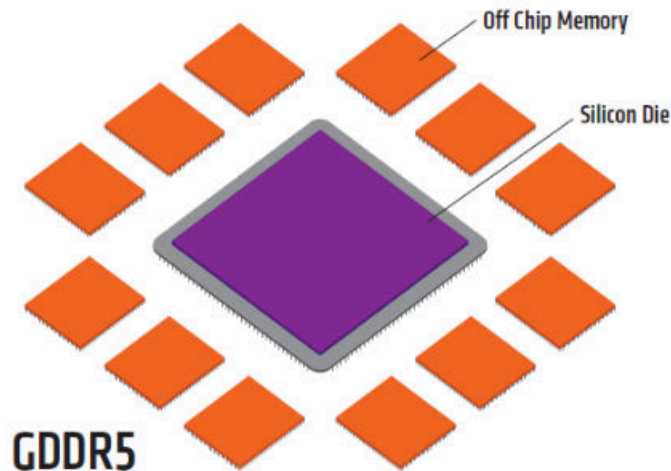
/ Deriving HBM



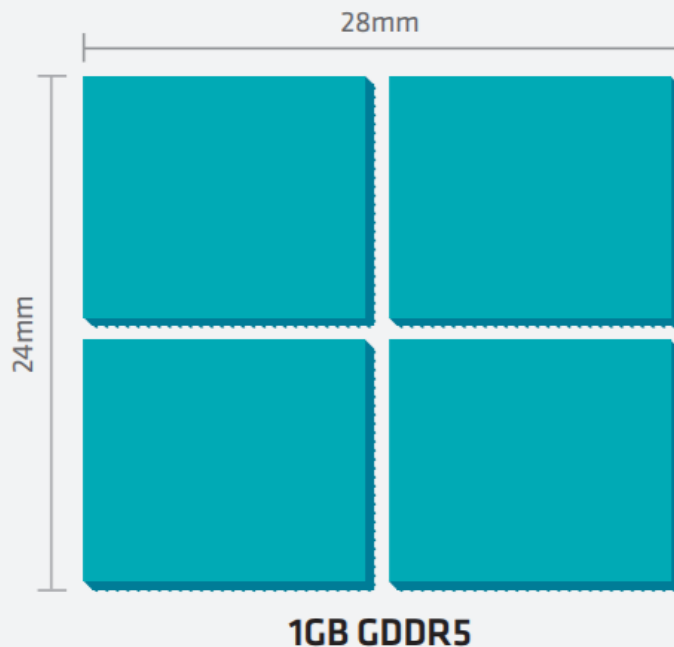
- 8x parallelism
- 8x bandwidth
- 1x capacity



HBM vs GDDR5: HBM shortens your information commute

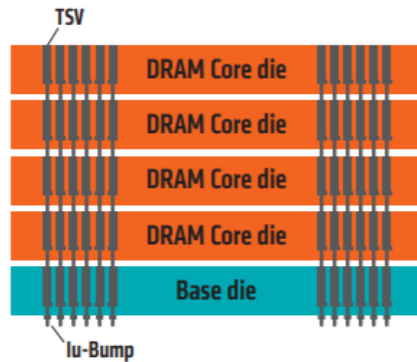
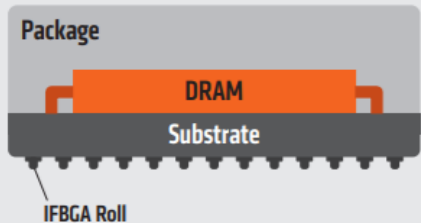


HBM vs GDDR5: Massive space savings



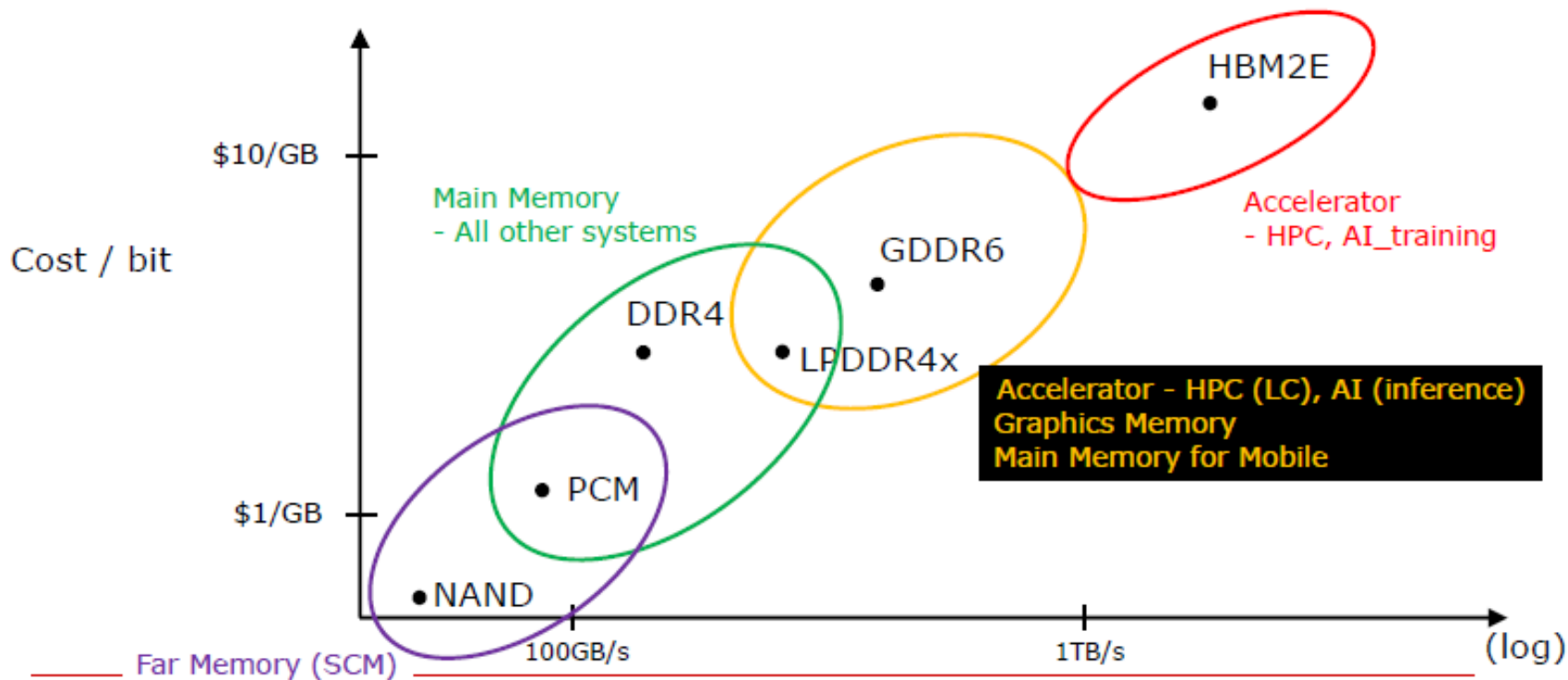
Areal, to scale

HBM vs GDDR5: Compare side by side



GDDR5	Per Package	HBM
32-bit	Bus Width	1024-bit
Up to 1750MHz (7GBps)	Clock Speed	Up to 500MHz (1GBps)
Up to 28GB/s per chip	Bandwidth	>100GB/s per stack
1.5V	Voltage	1.3V

/ Comparison



Review



- **A Detailed Look at Toolchain**
 - **Sticker**
 - **Optimization & Case Study**
- **Data Orchestration**
 - **Design Principles**
 - **Implicit vs Explicit**
 - **Coupled vs Decoupled**
- **Memory Subsystem Design**
 - **Memory metrics**
 - **Memory Technologies**

布置作业

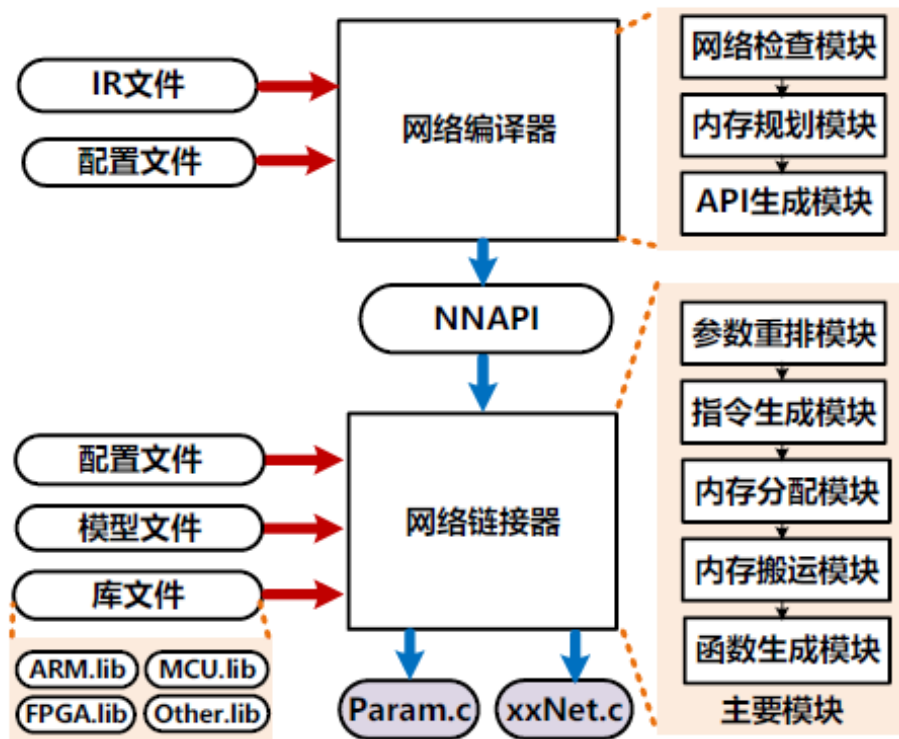


清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

Reading for this week

- 教材第4、5章, 软件架构、编程方法
- TVM论文

/ Review: Sticker工具链



- 网络编译器用于预处理文件
- 网络链接器用于划分计算方式以及连接库文件。

/ Review: Mapping中的切片优化

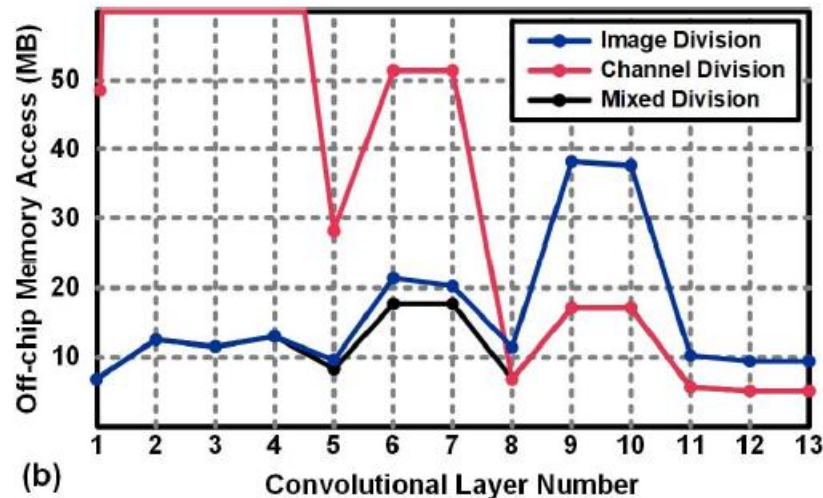


	Off-chip Access (W/O Compression Scheme)					
	Image Division		Channel Division		Mixed Division	
Input Image	$W_i \times H_i \times C_i$		$W_i \times H_i \times C_i$		$W_i \times H_i \times C_i$	
Output Image	$\frac{W_o \times H_o \times C_o}{\text{Pooling Size}}$		$\frac{W_o \times H_o \times C_o}{\times (\text{Ch. Div. \#} \times 2)}$		$\frac{W_o \times H_o \times C_o}{\times (\text{Ch. Div. \#} \times 2)}$	
Weight	$W_f \times H_f \times C_i \times C_o \times \text{Img. Div. \#}$		$W_f \times H_f \times C_i \times C_o$		$W_f \times H_f \times C_i \times C_o \times \text{Img. Div. \#}$	

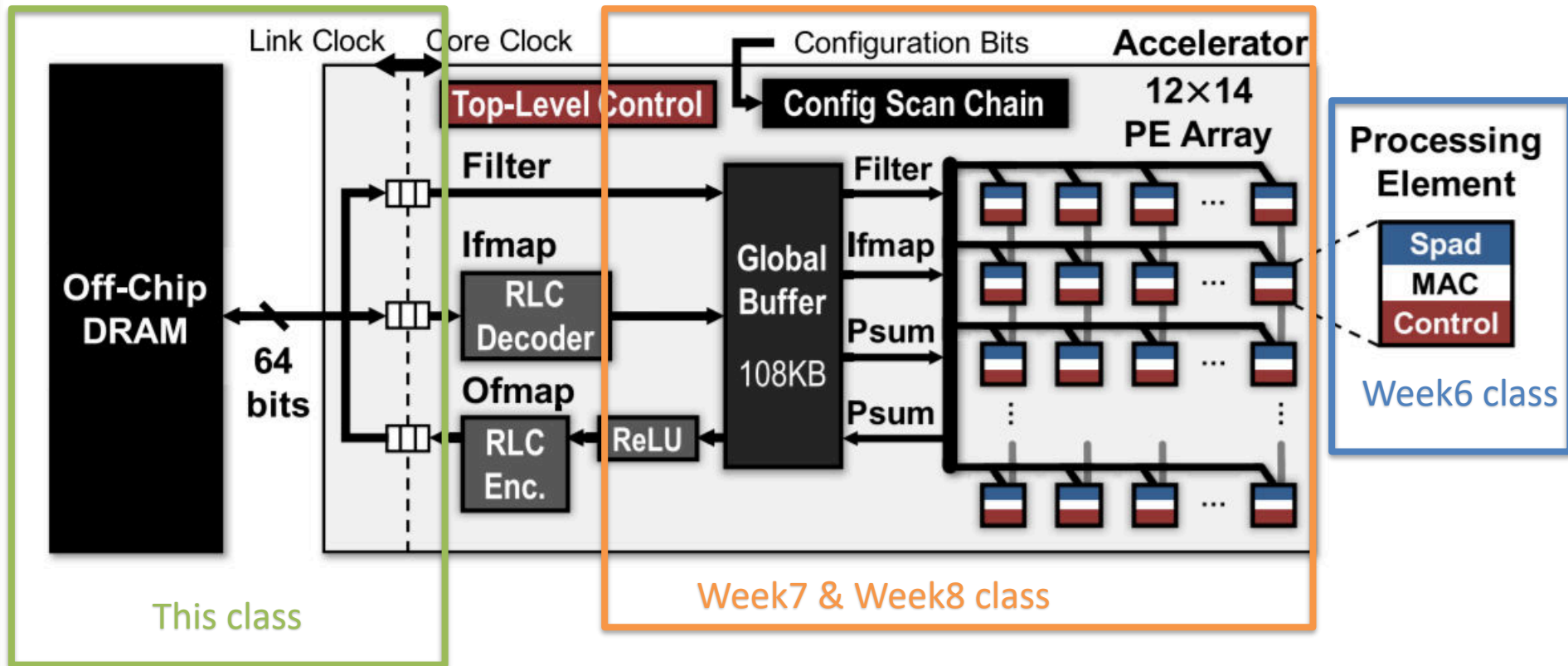
Img. Div. #	16	8	4	2	1
Ch. Div. #	1	2	4	8	16
Input Image	1.5	1.5	1.5	1.5	1.5
Output Image	1.5	6.1	12.3	24.5	49
Weight	18	9.0	4.5	2.3	1.1
Total (MB)	21.0	16.6	18.3	28.3	51.6

VGG-16 Example – 6th Convolution Layer

(a) $(W_i, H_i, C_i) = (56, 56, 256)$, $(W_o, H_o, C_o) = (56, 56, 256)$, $(W_f, H_f) = (3, 3)$

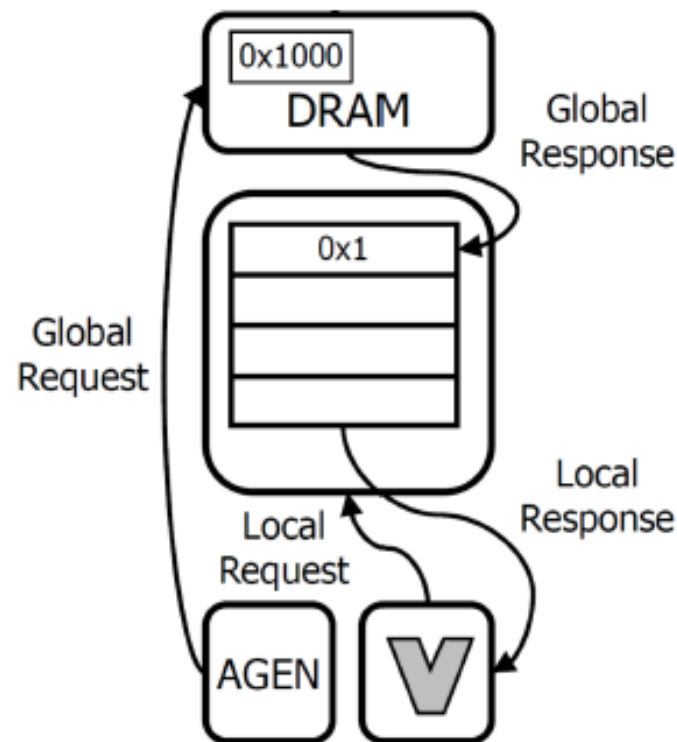
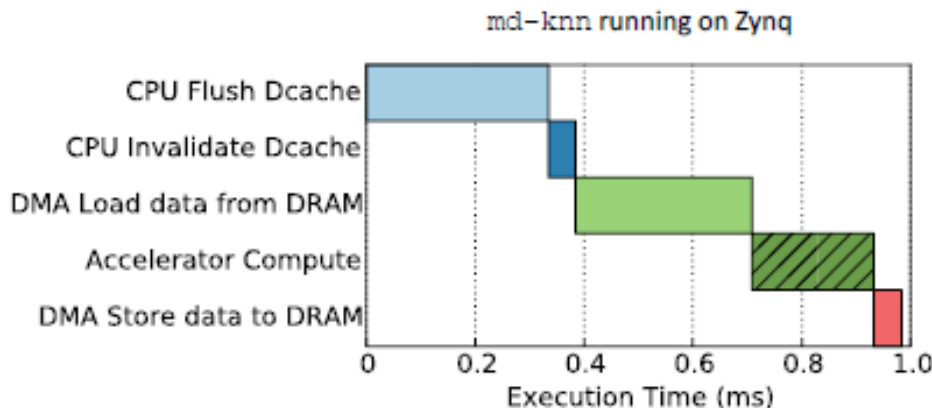


/ Review: A Typical AI Accelerator



/ Review: Domain-Specific Accelerators

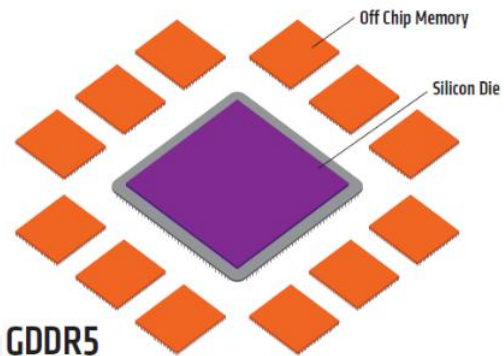
- Benefits:
 - Leverage static workload knowledge
 - Efficient hardware implementations
- Challenges:
 - Synchronization in a decoupled system



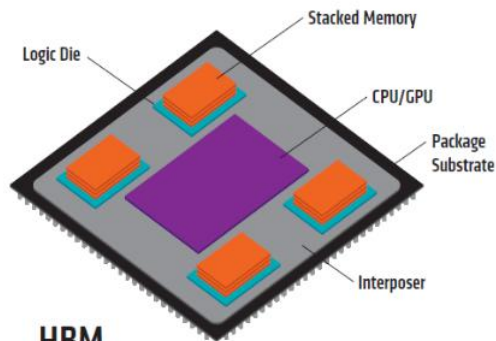
/ Review: HBM + AI Chip



清华大学电子工程系
Department of Electronic Engineering, Tsinghua University

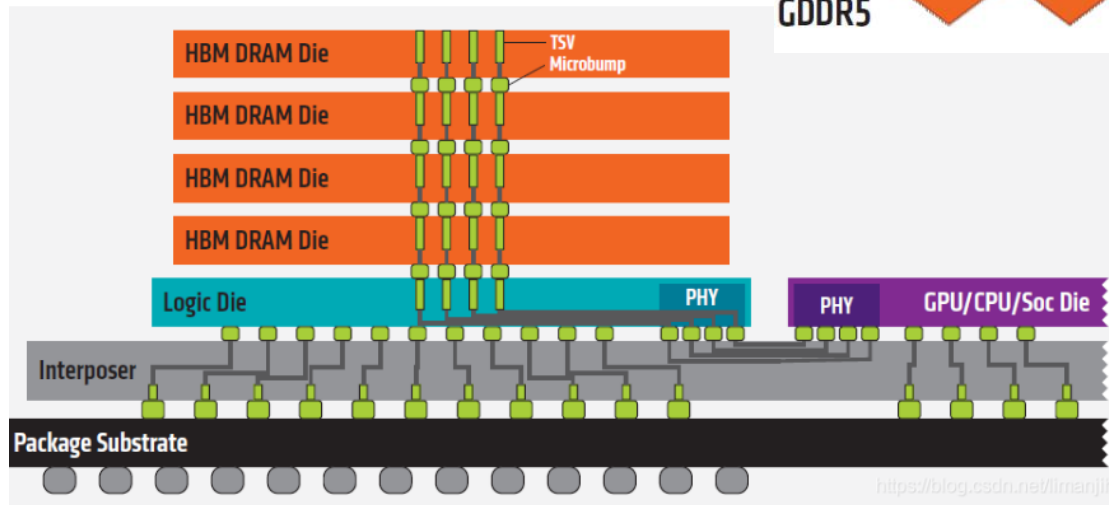


GDDR5



HBM

<https://blog.csdn.net/lianjiha>



<https://blog.csdn.net/lianjiha>